# BEHAVIOR AND REPRESENTATION IN OPEN-WEIGHT LARGE LANGUAGE MODELS FOR COMBINATORIAL OPTIMIZATION: FROM FEATURE EXTRACTION TO ALGORITHM SELECTION

ACCEPTED FOR PUBLICATION IN COMPUTERS & OPERATIONS RESEARCH

**Francesca Da Ros**
University of Udine, Italy
francesca.daros@uniud.it

**Luca Di Gaspero**
University of Udine, Italy
luca.digaspero@uniud.it

**Kevin Roitero**
University of Udine, Italy
kevin.roitero@uniud.it

arXiv:2512.13374v2 [cs.AI] 12 Aug 2026

**ABSTRACT**

Recent advances in Large Language Models (LLMs) open new perspectives for automation in optimization, yet little is known about whether their internal representations capture problem structure or algorithmic behavior. We investigate whether representations learned by frozen, open-weight LLMs for combinatorial optimization instances can support downstream decision tasks. The goal is not to replace exact feature extractors or to propose a new algorithm, but to assess whether such representations are reusable for feature recovery and algorithm selection. Our methodology combines *direct querying*, which tests explicit feature extraction, with *probing* analyses of whether this information is implicitly encoded in the hidden layers. The probing framework is further extended to a per-instance *algorithm selection* task. Experiments span four benchmark problems, three instance representations, and five open-weight models from 3B to 120B parameters across the Llama instruct and gpt-oss reasoning families, including a chain-of-thought study on Llama models. Results show a limited ability to recover features explicitly, particularly those requiring structured computation, while part of this information remains implicitly encoded in the hidden states. A consistent gap separates implicit encoding from explicit retrieval. Model scale attenuates this gap, whereas the effect of chain-of-thought prompting depends strongly on model size, and in all cases the gap remains open. Reasoning-oriented models also tend to abstain more when reliable computation is not possible. Notably, the predictive power of LLM hidden-layer representations is comparable to traditional feature extraction across all five models, indicating they can act as effective surrogate descriptors for downstream optimization tasks.

***Keywords*** Combinatorial Optimization · Large Language Models · Direct Querying · Probing · Feature Extraction · Algorithm Selection

## 1 Introduction

Combinatorial Optimization (CO) has long been used to tackle decision-making problems across many domains, including logistics, scheduling, and resource allocation [1–3]. Despite major advances in exact and heuristic methods, as well as the growing integration of Machine Learning (ML) techniques [4], developing and analyzing algorithms remains a largely manual, expertise-driven process. This process is time-consuming and difficult to automate, particularly when deep domain knowledge plays a critical role in performance.

In parallel, recent developments in Large Language Models (LLMs) have demonstrated their capacity to process text, generate executable code, and generalize across domains [5]. These capabilities have prompted researchers to explore how LLMs can support optimization-related tasks, from model formulation to heuristic generation and algorithmic analysis [6]. While many of these studies have produced encouraging results, they primarily assess the *functional performance* of the models (i.e., what they can do) without investigating their *representational understanding*, that is, whether their internal representations capture information about problem structure and algorithmic behavior.

Understanding how LLMs represent optimization problems is therefore particularly relevant. If LLMs encode meaningful information about problem structure or algorithmic performance, they could provide problem-independent representations for downstream tasks, rather than acting as conventional feature extractors engineered for specific problem formulations. Such representations could support algorithm selection, performance prediction, and hybrid optimization frameworks that combine learning and search. Conversely, the absence of such structure would highlight their current limitations in providing general-purpose representations for combinatorial optimization.

In this study, we investigate the extent to which LLMs capture information about COs, distinguishing between information encoded in their internal representations and information that can be explicitly recovered through direct querying. We focus on instance-level features and per-instance algorithm selection as complementary tests of problem characterization and algorithmic performance prediction. We address three research questions:

**RQ1** To what extent can LLMs identify salient structural or numerical characteristics of CO instances from their representations?

**RQ2** Can LLMs encode information about CO instances, and is this information accessible through probing?

**RQ3** Can representations extracted from the hidden layers of LLMs serve as effective instance features for per-instance algorithm selection, and how does their predictive power compare to that of traditional handcrafted features?

**RQ1** captures the behavioral dimension of our analysis, examining explicit feature awareness and generation capabilities. **RQ2** investigates the internal, latent encoding of instance-level information. **RQ3** assesses the practical utility of LLMs, testing whether their learned representations can inform algorithmic decision. To answer these questions, we combine *direct querying* and *probing* analyses.

Our analysis focuses on five open-weight LLMs spanning the Llama instruct and gpt-oss reasoning families in the 3B–120B parameter range, and all experiments are conducted on a stratified $5\%$ subset of the original benchmark collection, designed to preserve coverage across problems, feature complexity levels, and downstream labels while keeping the experimental campaign computationally feasible. Closed, commercial frontier models are excluded for reproducibility and cost reasons, since their behavior cannot be frozen across revisions of the study, their weights cannot be released alongside our artifact, and their per-token inference costs would be incompatible with the scale of our experiments.

In the first part of our study, we evaluate whether models exhibit awareness of instance-level features typically indirectly associated with algorithmic performance, and whether such features can be explicitly and implicitly retrieved from instance representations. We further analyze the role of explicit reasoning by comparing standard direct querying with Chain-of-Thought prompting, examining how its impact varies across model scales. In the second part, we use probing to examine whether the internal representations can act as surrogates for handcrafted features traditionally obtained through explicit feature engineering. We conduct experiments across four representative Combinatorial Optimization Problems (COPs), using publicly available instance sets and algorithm portfolios from prior Instance Space Analysis (ISA) studies. Our evaluation spans multiple feature complexities, instance representations, model scales and prompting strategies, and probing configurations, providing a comprehensive perspective on the relationship between explicit and implicit representations in LLMs.

The central hypothesis of this work is that explicit numerical recovery and latent representational usefulness are distinct properties. An LLM may fail to compute or return an exact feature value when prompted, while its hidden states may still contain a feature-related signal that is useful for downstream prediction. This distinction is central to our analysis and motivates the joint use of direct querying, probing, and algorithm-selection experiments.

The main contributions of this work are as follows:

- We introduce a unified experimental framework for evaluating what LLMs capture about COPs through direct querying and probing.
- We validate the framework across five open-weight models spanning the 3B–120B range, and analyze the role of explicit reasoning through a Chain-of-Thought study on Llama models, showing that its effect depends strongly on model scale.
- We provide the first large-scale analysis of how LLMs encode structural, numerical, and algorithmic information across multiple problem domains and input representations.
- We show that explicit retrieval and latent encoding do not fully coincide: representations may support downstream prediction even when direct querying fails to reliably recover feature values.

- We release the dataset, code, and prompts to foster reproducibility and further exploration at the intersection of LLMs and optimization.[1]

This study extends our earlier conference publication [7], which focused solely on direct querying of LLMs for feature extraction. Here, we expand the analysis along several dimensions: we introduce probing analyses to explore internal representations, generalize the experimental setup to multiple problems and input representations, include a downstream algorithm-selection task, and provide a broader comparative analysis; additionally, we extend the empirical validation from one to five open-weight models and evaluate, on a stratified $5\%$ subset of the benchmark collection, multiple prompting strategies beyond direct querying, including Chain-of-Thought (CoT) prompting and native reasoning.

Accordingly, the study is positioned as a representation-analysis framework rather than as a solver-design contribution: it investigates whether frozen LLM representations can serve as problem-agnostic instance descriptors and compares their predictive value with handcrafted and ISA-based features in a standard algorithm-selection pipeline.

The remainder of this paper is organized as follows. Section 2 reviews the background and related work. Section 3 describes the proposed methodology. Section 4 presents the experimental analysis whereas Section 5 interprets the findings under a unified framework. Finally, Section 6 concludes the paper and outlines directions for future research.

## 2 Background and Related Work

This section reports background and related work on the topics of LLM (Section 2.1) and their interpretability (Section 2.2). Following, it summarizes relevant notions regarding LLM usage in CO (Section 2.3) and algorithm selection (Section 2.4).

### 2.1 Large Language Models

LLMs have fundamentally transformed the landscape of Natural Language Processing (NLP) and, increasingly, fields beyond it, including reasoning and programming [8, 9].

LLMs are Neural Network (NN) architectures designed to model the probability distributions of natural language sequences. The fundamental processing unit of an LLM is the token, namely a sub-word or symbol representing a fragment of text, code, or other structured input. Given a sequence of *tokens* $t_1, t_2, \ldots, t_n$, an LLM estimates the conditional probability $P(t_n|t_1, \ldots, t_{n-1})$, thereby learning to predict the next token conditioned on its preceding context. Through large-scale training on heterogeneous textual and symbolic corpora, such models acquire internal representations that capture syntactic, semantic, and relational regularities, which in turn enable them to perform a broad spectrum of reasoning, generation, and comprehension tasks [10, 11].

At their core, most LLMs implement the Transformer architecture [12], which relies on self-attention mechanisms to compute contextual dependencies between all tokens in parallel [13]. This design allows LLMs to handle long-range relationships efficiently and to generate outputs that are sensitive to the entire input sequence.

Users interact with the model through a prompt, which is a textual sequence that defines the input context and often specifies the desired task. The model then produces one or more output tokens that, when decoded, yield natural language or symbolic text. The process can be guided or restricted through prompt engineering or constrained decoding, the latter enforcing structural, syntactic, or semantic constraints on the generated output, e.g., requiring valid JSON, enforcing type consistency, or limiting values to predefined domains.

### 2.2 Large Language Models Interpretability

Despite their remarkable capabilities, the internal mechanisms by which LLMs encode and organize knowledge remain only partially understood. Two complementary approaches are commonly employed to investigate these mechanisms: *direct querying* (also referred to as *direct questioning* or *prompting*), which analyzes model behavior through carefully designed prompts, and *probing*, which examines the information embedded within the model hidden representations.

Direct querying is a methodology used to assess the behavior of LLMs by presenting them with a prompt and analyzing their generated responses. It evaluates the model's ability to explicitly retrieve information when prompted, without inspecting internal activations or mechanisms. A well-known example of direct querying is the *needle in a haystack* task,[2] in which a factual statement (i.e., a short text fragment) is embedded within a long context, and the model

[1] Code and data: https://doi.org/10.5281/zenodo.21891840.

[2] https://github.com/gkamradt/LLMTest_NeedleInAHaystack/tree/main, accessed 14 Oct 2025.

is prompted to retrieve it *verbatim*. This methodology has been extended to other applications, such as document summarization and citation generation [14], to evaluate whether relevant information can be surfaced by the model when explicitly prompted to recall it.

Probing offers a systematic and empirically grounded framework for investigating the internal representations of LLMs. It provides a structured methodology to examine what kinds of information are captured within the models' internal mechanisms, where such information is stored, and how it is organized and retrieved across layers [15–17]. Unlike the output-based evaluation described above, which focuses on model behavior elicited through direct querying, probing operates by extracting the hidden layer activations produced as the model processes an input, and analyzing these activations using lightweight predictive models known as *probes*. This approach enables the assessment of whether specific properties, including syntactic, semantic, numerical, and domain-specific features, are encoded in the model representations and can be recovered by a downstream model.

A typical probing setup comprises three main stages: *(i)* a controlled set of inputs is fed into a frozen LLM to collect activations from one or more internal layers of interest, typically the final layer; *(ii)* an external model (i.e., the probe) is trained on these representations to predict a target property; and *(iii)* the probe's performance is evaluated on unseen data to assess its generalization on the same property. Common probe architectures include linear classifiers and simple Multi-Layer Perceptrons (MLPs), each suited to investigating different aspects of how information is encoded within LLMs [18–20]. Simpler probes, such as linear regressors, are typically used to examine whether a property is linearly accessible from the model representations, whereas more complex, non-linear probes can explore relationships that may be distributed or non-linearly structured within the latent space [21].

Although originally developed for linguistic analysis, probing has been applied to a broad range of interpretability tasks. Several studies have examined whether hidden layers encode syntactic categories or dependency relations, showing that intermediate layers tend to capture structural information, whereas deeper layers increasingly specialize in semantic content [18, 22]. The same framework has also been extended to investigate higher-level capabilities such as discourse awareness [23], factual and relational knowledge [24], and hierarchical reasoning [25]. For instance, a probe may be trained to distinguish correct from incorrect factual triples (e.g., Paris is the capital of France vs. France is the capital of Paris) or to recover the latent coordinates of entities mentioned in text, thereby revealing whether spatial or relational concepts are implicitly represented within the model. Such applications show that probing can provide insights into the organization of latent representations in LLMs, extending beyond surface-level linguistic features.

A key advantage of probing lies in its architecture-agnostic nature: it treats the model as a frozen feature extractor and requires neither fine-tuning nor parameter updates. Consequently, it ensures that the subsequent analysis based on the model weights accurately reflects its learned internal state [26].

### 2.3 Large Language Models for Combinatorial Optimization

Several recent studies have investigated the use of LLMs in CO [6, 27, 28]. Early interest in combining LLMs with CO emerged from the Natural Language for Optimization (NL4Opt) competition [29]. It explored whether NLP techniques could assist in mathematical modeling through entity recognition and automatic problem formulation, initially focusing on Linear Programming (LP) tasks. Building upon this, subsequent research has diversified along two main directions: *(i) Model definition and formulation*, where LLMs generate or translate optimization models from textual descriptions [30]; and *(ii) Heuristic and algorithm construction*, where LLMs design, adapt, or parameterize search procedures [31].

With respect to model formulation, recent works have extended beyond LP to encompass other paradigms such as Constraint Programming (CP) [32, 33] and Mixed Integer Linear Programming (MILP) [34]. In the second direction, applications include the automated discovery of novel heuristics, as exemplified by FunSearch [35], although Sim et al. [36] later showed that several simple, classical heuristics outperform those generated by the model across extensive benchmark experiments. Alternative approaches focus on generating and evolving metaheuristics components, as in LLaMEA [37], or on enhancing existing algorithms [38].

A smaller body of research has explored supportive and analytical roles of LLMs within CO, extending beyond problem solving. For instance, Sartori et al. [39] employed LLMs to assist users in interpreting and navigating Search Trajectory Networks (STNs), which are graph-based structures that capture the temporal evolution of (meta-)heuristic search processes [40].

Despite the growing body of research, most existing studies prioritize functional performance over representational or explanatory understanding. Notable exceptions include the analysis of LLM-generated code through graph-based representations [41] and behavioral investigations of generated heuristics using STNs [42]. However, how LLMs

internally encode combinatorial structure, and whether such representations can be effectively accessed or exploited, remains largely unexplored. The limited number of systematic examinations of their hidden representations leaves a substantial gap between observable competence and latent understanding. This work aims to bridge this gap by investigating not only how combinatorial structures are represented within LLMs, but also how such information can be accessed through direct querying and probing, and whether it can be effectively leveraged in downstream tasks such as algorithm selection.

### 2.4 Algorithm Selection in Combinatorial Optimization

Within CO, algorithmic performance varies substantially across instances: an algorithm that performs well on one subset of instances may be ineffective on another, depending on factors such as instance size, structure, or constraint tightness. This variability arises from the inherent complexity of most COPs, which are NP-hard and exhibit widely differing computational characteristics across instances. The phenomenon is formalized by the No Free Lunch (NFL) theorem [43], which states that no single algorithm can achieve superior performance across all possible problems or instance distributions.

The challenge of selecting or configuring algorithms according to instance characteristics was systematically conceptualized by Rice [44], who defined the Algorithm Selection Problem as the task of learning a mapping between instance features and algorithm performance. In Rice's framework, each problem instance is represented by a vector of measurable characteristics, and each algorithm is evaluated according to one or more performance metrics; the objective is to predict which algorithm is expected to perform best for a given instance, effectively framing the problem as a supervised learning task. This formulation laid the foundation for per-instance algorithm selection and inspired subsequent developments in algorithm portfolios: collections of complementary algorithms designed to exploit performance diversity across instances [45, 46].

Successful algorithm selection increasingly relies on quantitative instance characterization, that is, the identification of features that capture the structural and behavioral properties of problem instances. To support this process, frameworks such as ISA [47] and tools such as MATILDA[3] have been developed, offering integrated environments that go beyond algorithm selection to include feature extraction, visualization, and mapping of algorithm performance across low-dimensional instance spaces [48, 49].

Despite these advances, feature extraction still relies heavily on handcrafted, problem-specific descriptors that demand substantial domain expertise. To address this limitation, recent research has explored automated feature learning, where models learn informative representations directly from data. For instance, NNs have been employed to infer latent instance representations [50, 51], extending also to black-box optimization settings [52]. Our work differs both in scope and realization: to the best of our knowledge, this is the first study to systematically evaluate LLMs as feature extractors for combinatorial optimization, combining direct querying, probing, and downstream validation across multiple problems, representations, and model scales. It is also the first to investigate whether internal LLM layers can serve as predictors of algorithmic performance, thereby linking latent model representations to algorithm-selection outcomes.

## 3 Methodology

In this section, we present the methodology adopted to investigate how LLMs represent and process COPs, with the broader goal of assessing whether these models encode information that can support algorithm selection. Specifically, our analysis is structured around two complementary perspectives: *(i)* the *explicit recoverability* of instance-level information, i.e., whether the models can retrieve target features when prompted; and *(ii)* their *implicit representations*, i.e., the extent to which internal layers encode information about problem instances and their properties. To address these perspectives, we employ a twofold experimental strategy: *(i) Direct querying*, wherein models are explicitly prompted to infer instance-level features; and *(ii) Probing*, wherein the models internal activations are analyzed to assess whether features or algorithmic performance signals are implicitly represented.

We remark that a key design choice in this study is the exclusive use of off-the-shelf LLMs without any form of task-specific fine-tuning. Thus, the objective of this work is to assess whether these models, as released, already encode meaningful representations of combinatorial optimization problems, and whether such representations can support downstream tasks such as feature extraction and algorithm selection. Introducing fine-tuning would shift the focus from analyzing intrinsic model behavior to evaluating model adaptation, opening a broad and orthogonal set of research questions, including the choice of supervision methods, fine-tuning strategies (e.g., supervised fine-tuning or reinforcement learning approaches), and data construction. Moreover, fine-tuning requires the selection of

[3] https://matilda.unimelb.edu.au/matilda/, accessed 13 Oct 2025.

Table 1: Overview of the methodological components of this study. Each component addresses a specific research goal and employs a dedicated evaluation strategy.

| Goal | Methodology | Evaluation | Section |
|---|---|---|---|
| Assess whether LLMs can explicitly identify problem features from instance representations | **Direct Querying**: models are prompted to extract features directly from the instance | Comparison between model-predicted and ground-truth features | Section 3.2 |
| Determine whether internal representations of LLMs encode problem features | **Feature Probing**: regressors are trained on last-layer activations to predict instance features | Comparison between predicted and ground-truth features | Section 3.3 |
| Determine whether internal representations of LLMs encode information predictive of algorithm performance | **Algorithm Selection Probing**: classifiers are trained on last-layer activations to predict the per-instance best algorithm | Classification accuracy, comparison with baseline methods | Section 3.4 |

task-specific datasets, which may introduce additional biases and confound the analysis of what the model captures prior to adaptation. For these reasons, fine-tuning is intentionally outside the scope of this work and is left for future investigation.

We begin by outlining the CO setting, including the problems considered, the instance representations, the feature sets, and the algorithm portfolios (Section 3.1). We then describe the direct querying procedure (Section 3.2), followed by two probing analyses: the first focusing on the representation of problem features (Section 3.3), and the second on information relevant to algorithm selection (Section 3.4). The overall structure of our methodology is summarized in Table 1, which provides an overview of the specific objectives, approaches, and evaluation procedures associated with each experimental component.

### 3.1 Combinatorial Optimization Concepts

We recall here the main concepts and definitions underlying CO to establish a common terminology and clarify how these notions are instantiated in this study.

Optimization problems aim to identify the best solution(s) from a search space, i.e., the set of feasible solutions, by optimizing one or more objective functions subject to a set of constraints. Each solution corresponds to an assignment of values to a collection of decision variables. In CO, these decision variables take discrete values, resulting in a finite, yet typically exponentially large, search space. Such a formulation encompasses both well-known benchmark problems, such as the seminal Maximum Satisfiability Problem (max-SAT), and a variety of real-world applications [53–55].

Each COP defines a general problem structure that is instantiated through specific *instances*, which provide the concrete input data. For example, the Graph Coloring Problem (GCP) is defined on an undirected graph $G = (V, E)$, where $V$ denotes the set of nodes and $E$ the set of edges connecting pairs of nodes. The task is to assign a color to each node such that adjacent nodes receive different colors, yielding a valid coloring that minimizes the total number of colors used. An instance of the GCP therefore specifies the graph topology, i.e., the nodes and edges that define $G$. Instances are typically stored in standardized formats and parsed into data structures suitable for algorithmic processing.

For example, GCP instances are available in the DIMACS format,[4] an abridged description of which is reported below:

> In this format, nodes are numbered from 1 up to $n$. There are $m$ edges in the graph.
> Files are assumed to be well-formed and internally consistent: node identifier values are valid, nodes are defined uniquely, exactly $m$ edges are defined, and so forth.
> Comment lines give human-readable information about the file and are ignored by programs. Comment lines can appear anywhere in the file. Each comment line begins with a lower-case character c. There is one problem line per input file.
> The problem line must appear before any node or arc descriptor lines. The problem line has the following format: p FORMAT NODES EDGES. The lower-case character p signifies that this is the problem line. The FORMAT indicates the graph format. The NODES field contains an integer value specifying the number of nodes in the graph. The EDGES field contains an integer value specifying the number of edges in the graph.
> There is one edge descriptor line for each edge in the graph, each with the following format: e $u$ $v$. The lower-case character e signifies that this is an edge descriptor line. For an edge $(u, v)$ the fields $u$ and $v$ specify its endpoints. Each edge $(u, v)$ appears exactly once in the input file and is not repeated as $(v, u)$.

An instance in this representation for the GCP is illustrated below:

[4] http://archive.dimacs.rutgers.edu/, accessed 14 Oct 2025.

Table 2: Graph Coloring Problem features.

| Name | Description | Compl. |
|---|---|---|
| `feat_nodes` | The total number of nodes in the graph. | ○ |
| `feat_edges` | The total number of edges in the graph. | ○ |
| `feat_degree_1` | The degree of node 1 in the graph, representing the number of connections per node. | ● |
| `feat_density` | The density of the graph, calculated as the ratio of 2 times the number of edges to the number of nodes times the number of nodes minus 1. | ● |
| `feat_ratio_1` | The ratio of the number of nodes to the number of edges in the graph. | ● |
| `feat_ratio_2` | The ratio of the number of edges to the number of nodes in the graph. | ● |
| `feat_degree_mean` | The average degree of the nodes in the graph, representing the mean number of connections per node. | ● |
| `feat_degree_max` | The maximum degree of the nodes in the graph, indicating the highest number of connections any node has. | ● |
| `feat_degree_min` | The minimum degree of the nodes in the graph, indicating the lowest number of connections any node has. | ● |

```
p edge 100 1902
e 1 2
e 1 6
e 1 10
...
```

In the remainder of this work, we refer to this standardized format as standard. In addition to it, we also consider two alternative instance representations: natural language descriptions (natural language) and code-like formulations derived from the MiniZinc modeling language (code-like). The same instance introduced above can be expressed in natural language form as follows:

> The instance is named name. The graph has 100 nodes and 1902 edges. There is an edge between node 1 and node 2. There is an edge between node 1 and node 6. There is an edge between node 1 and node 10. ...

The corresponding code-like format is:[5]

```
int: n = ...;
set of int: V = ...;
int: num_edges = ...;
array[1..num_edges, 1..2] of V: E = ...;
```

Problem instances can be further characterized through *features*, i.e., measurable attributes that capture structural or statistical properties of an instance and are often problem-specific. Features enable comparisons across instances and support tasks such as ISA and algorithm selection. In this study, features are considered in two complementary ways:

1. **Feature extraction and interpretation**: we investigate whether, and to what extent, LLMs can identify or reconstruct relevant instance features directly from the instance representations. Features are grouped by their extraction complexity:[6] *(i) directly extractable features*, which appear explicitly in the instance (○); *(ii) low-effort features*, which require minimal computation, such as counts or extrema (◓); and *(iii) high-effort features*, which involve aggregation or derived calculations, such as averages (●). Table 2 reports examples of features used for the GCP.
2. **Baseline for algorithm selection**: we use both the above features and additional descriptors defined in prior ISA studies [48, 56–58] as inputs to traditional per-instance algorithm selection models. These baselines serve as reference points against which we compare models that instead rely on LLM hidden-layer representations.

COPs are typically solved by *algorithms* that explore the search space to identify high-quality solutions. These include *exact methods*, which guarantee optimality but are often computationally prohibitive, and *(meta-)heuristic methods*, which trade optimality for efficiency and scalability. In this study, we adopt an *algorithm portfolio* perspective, considering sets of algorithms with complementary performance across instances. Algorithmic performance is evaluated by comparing the objective values obtained on each instance.

For illustration, in the case of the GCP, we consider two graph-coloring heuristics, DSATUR (degree saturation) and MAXIS (maximal independent set), which have been demonstrated to exhibit complementary strengths and weaknesses through ISA [59]. The performance metric is the raw objective value, namely the number of colors in a valid coloring, computed on the same set of benchmark instances used in that study [59].

Figure 1 provides a schematic overview of this methodological framework, illustrating how the main CO concepts are organized and instantiated across the case studies considered in this work. Specifically, for each problem, we

[5] https://www.hakank.org/minizinc/coloring_ip.mzn, accessed 15 Oct 2025.

[6] Extraction complexity is defined relative to the LLM, reflecting how difficult a feature is for the model to infer.

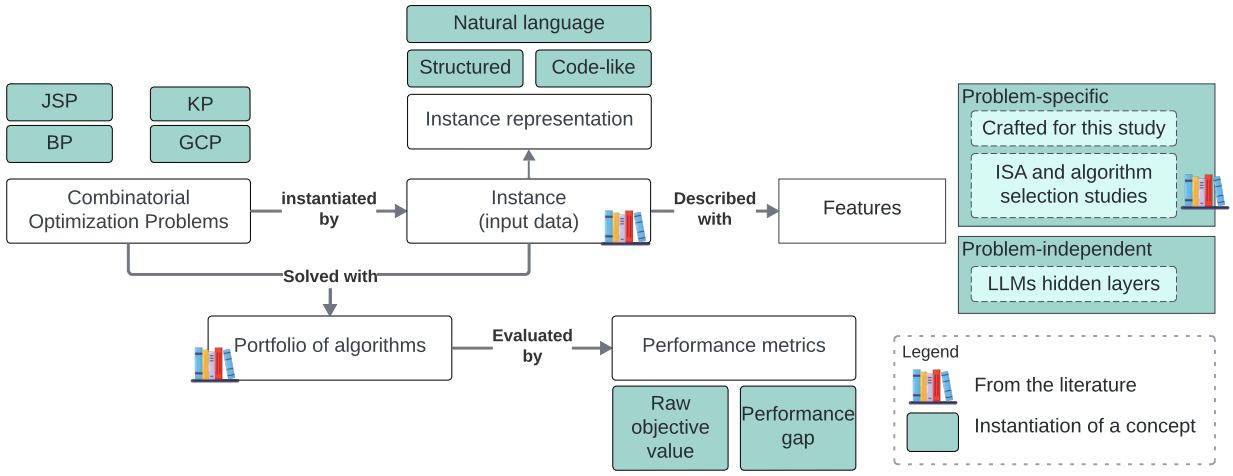



Figure 1: Conceptual structure of the case studies and their instantiation in this work.

retrieve benchmark instances from prior ISA studies to ensure diversity in structural characteristics. Each instance is expressed through different representations, either structured (e.g., standard or code-like) or natural language (natural language). Moreover, each instance is characterized by two complementary types of features. The first are *handcrafted features*, conceived, designed, and extracted through manually written code, as in traditional algorithm selection analyses. The second are *automatically derived features*, obtained directly from LLM hidden-layer activations without the need for explicit feature engineering. Quantitative characterization of the instances relies on these feature sets, while algorithm portfolios comprise methods identified in the literature as exhibiting complementary performance, evaluated through standard metrics such as raw objective value and performance gap.

Table 3 summarizes the main characteristics of the case studies considered in this work. We selected four COPs that cover different combinatorial structures and search spaces (assignments, permutations, and graphs): namely, the Bin Packing Problem (BPP), Graph Coloring Problem (GCP), Jobshop Scheduling Problem (JSP), and Knapsack Problem (KP). These are well-established benchmark problems and are therefore not described here in detail. The corresponding instances, together with their ISA features and algorithm performance data used for algorithm selection, are available on the MATILDA[7] website. For each COP, we report the number of benchmark instances, the number of algorithms included in the portfolio, and the number of features considered (— means no feature of that type). The column ISA refers to the study from which the instances, algorithm portfolios, and the features labeled as ISA were originally derived. The columns STANDARD DESC. and CODE-LIKE indicate the source of the standardized and code-like representations, respectively.

### 3.2 Direct Querying of Large Language Models

Direct querying is employed to assess the extent to which LLMs can identify or describe relevant characteristics of COPs based on given instance representations. This approach focuses on externally observable behavior. In this study, we consider two prompt variants within the direct-querying setting and three overall querying configurations. The prompt variants are *standard direct querying*, also referred to as zero-shot querying, in which the model is asked to return only the target numeric value, and *Chain-of-Thought direct querying*, in which the prompt explicitly requests a short reasoning trace before the final structured output. In addition, for gpt-oss models we consider a *native reasoning* configuration implemented through the Harmony interface: in this case, the model receives the same user-facing prompt as in the standard variant, but reasoning is produced through the model's native analysis channel under a fixed developer instruction, a controlled reasoning-effort setting, and the same structured-output constraints.

Prompts are constructed according to the ReasoningTaskFormat (RTF) framework to ensure consistent task framing and output structure across all models. Each prompt comprises three components:

- **Reasoning preamble:** defines the assumed role/domain expertise;
- **Task description:** specifies the reasoning objective or operation to be performed;

[7] https://matilda.unimelb.edu.au/matilda/, accessed 13 Oct 2025.

Table 3: Summary of the considered problems.

| Problem | Instances (selected) | Algorithms | Features for the direct querying and probing extraction | | | | | | ISA | Standard desc. | Code -like |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | List | Type | | Compl. | | | | | |
| | | | | Int | Float | ○ | ◓ | ● | | | |
| BP | 8,815 (440) | 2 | Bin capacity, items, maximum/minimum/average item weight | 4 | 1 | 2 | 2 | 1 | [60] | | |
| GCP | 8,278 (413) | 2 | Nodes, edges, graph density, ratio of the number of nodes to the number of edges in the graph and vice versa, degree of the first node, maximum / minimum / average degree | 5 | 4 | 2 | — | 7 | [59] | [61] | |
| JSP | 4,467 (223) | 12 | Jobs, machines, operations, maximum / minimum / average duration | 5 | 1 | 2 | 3 | 1 | [62] | | [63] |
| KP | 5,300 (265) | 3 | Capacity, maximum / minimum / average weight/profit, weight / profit / efficiency of the first item, average efficiency | 7 | 4 | 1 | 7 | 3 | [58] | | [63] |

- **Format specification:** constrains the response to a predefined structure, ensuring syntactic validity and facilitating automated evaluation.

Both prompt variants share the same problem statement, feature specification, and instance input, and differ only in the output instructions. The standard variant asks the model to return the target value directly in JSON format, whereas the Chain-of-Thought variant asks for a short reasoning trace followed by the same target value. Native reasoning in gpt-oss models uses the same user-facing prompt as the standard variant, but differs in the interaction protocol and in the availability of an internal reasoning channel. This design allows us to isolate the effect of explicit intermediate reasoning while keeping the task specification unchanged.

The standard direct-querying prompt template used in this study is presented below. It shows how the reasoning preamble, task description, and format specification are combined.

```
You are given an instance of a combinatorial optimization problem: {problem_name}.
  This is not a coding task. Do not return any code.
  Extract the numeric value of the following feature from the instance:
  - Feature name: {feature_name}
  - Feature description: {feature_description}
  - Expected type: {feature_type}
  The instance is provided here: """
  {instance}
  """
  Instructions:
  - Return a JSON object only, with a single field "value", i.e., `{{ "value": ... }}`.
  - The "value" field should contain the numeric value of the required feature
    and should be of the expected type (i.e., {feature_type}).
  - If the value is unknown/undeterminable, return `{{ "value": null }}`.
  - No explanations, no extra fields.
  Answer:
```

The Chain-of-Thought prompt variant retains the same task description and instance input, but modifies the instructions so that the model first provides a short explanation of how the value is derived from the raw problem data and then returns the target value in JSON format. The corresponding template is reported below:

```
You are given an instance of a combinatorial optimization problem: {problem_name}.
  This is not a coding task. Do not return any code.
  Extract the numeric value of the following feature from the instance:
  - Feature name: {feature_name}
  - Feature description: {feature_description}
  - Expected type: {feature_type}
  The instance is provided here: """
  {instance}
  """
  Note: the instance contains only raw problem data  feature names do not appear
  as labels inside it. You must derive the value by reading and interpreting the
  data structure directly.
  Instructions:
  - Before returning the JSON, reason step by step (2-4 sentences) about how you
    identified the value from the raw data structure, not from any label or name.
  - Return a JSON object with exactly two fields:
    - "reasoning": your concise step-by-step explanation.
    - "value": the numeric value of the required feature, of type {feature_type}.
  - If the value is unknown or undeterminable, set "value" to null.
  - No extra fields beyond "reasoning" and "value".
  Answer:
```

For gpt-oss models, direct querying is implemented through the Harmony message format, using three messages: a system message specifying the reasoning effort, a fixed developer instruction enforcing precise JSON-only extraction, and a user message containing the same task specification used in the standard direct-querying prompt. This setup enables native reasoning while preserving the same user-facing task formulation adopted for the other models.

Additionally, the LLM is equipped with a structured decoding layer,[8] which guides the generation process and constrains the output to follow a predefined JSON schema. This mechanism ensures syntactic validity and type consistency with the feature being predicted. Two schemas are used, one for each prompting variant. In the standard direct-querying setting, the output contains a single field `value`. In the Chain-of-Thought setting, the output contains exactly two fields, `reasoning` and `value`. In both cases, evaluation is performed exclusively on the `value` field, ensuring direct comparability between the two prompting variants.

```
{
  "type": "object",
  "additionalProperties": false,
  "required": ["value"],
  "properties": {
    "value": { "anyOf": [ { "type": "[feature-type]" }, { "type": "null" } ] }
  }
}
```

For the Chain-of-Thought variant, the schema is added with the additional required field `"reasoning":{"type":"string"}`.

The correctness of model-generated features is evaluated by comparing them against ground-truth values computed from the corresponding instances, using both exact-match and error-based criteria. As for the Chain-of-Thought variant, only the numeric `value` field is considered in the quantitative evaluation; the `reasoning` field is retained as a behavioral artifact of the prompting strategy, but is not scored directly.

The overall setup of the direct-querying experiment is depicted in Figure 2. For each problem, the LLM receives a prompt reporting an instance in one of three formats together with a textual instruction specifying the target feature. Depending on the prompt variant and execution mode, the model either returns the target value directly (standard direct querying), produces a short reasoning trace followed by the same target value (Chain-of-Thought direct querying), or generates the final answer through a native reasoning channel while preserving the same user-facing task specification (native reasoning with gpt-oss models). The direct-querying procedure is repeated for all features defined for each problem. Predicted feature values are then compared with ground-truth values obtained by running handcrafted feature extractors on the same instance.

Generation budgets are adjusted to the prompt variant and to the underlying interaction format: 64 tokens for standard direct querying with instruct models, 512 for Chain-of-Thought prompting, and 1,024 for native reasoning. For gpt-oss models, a minimum budget of 512 tokens is enforced even in the standard setting, since the Harmony interface opens an internal analysis channel before the final JSON output.

[8] https://blog.vllm.ai/2025/01/14/struct-decode-intro.html, accessed 17 Oct 2025.

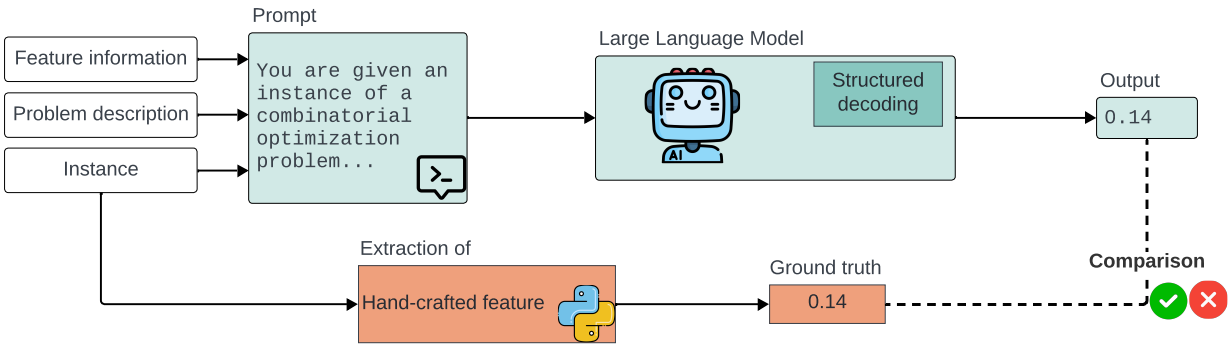



Figure 2: Overview of the direct querying pipeline.

### 3.3 Probing of Large Language Models

Probing is employed to investigate whether LLMs encode information about the structural and numerical characteristics of CO instances. In contrast to direct querying (Section 3.2), which analyzes externally observable behavior, probing examines the latent representations learned by the model, providing a complementary perspective on how instance properties are internally encoded.

For each combination of problem and instance representation, every instance is processed by the model without triggering text generation. Hidden-layer activations are extracted from the final layer, and a single vector representation for each instance is obtained by applying three common pooling strategies: *mean*, *max*, and *last* (see, e.g., Tang and Yang [64]). The *mean* strategy computes the average of the last hidden-layer activations across all tokens; the *max* strategy takes the maximum activation along each representation dimension across tokens; and the *last* strategy uses the activation vector corresponding to the End-Of-Sequence (EOS) token, i.e., the model final prediction signal indicating the termination of generation. These pooled vectors provide compact, contextualized embeddings of each instance, which are then used as input to downstream probing models to assess whether the model internally encodes instance-level structural and numerical characteristics.

Each pooled vector is then paired with the ground-truth feature values corresponding to the same instance, forming a dataset for supervised probing. For each feature and representation, we train a separate regression model (probe) to assess whether the feature can be accurately decoded from the hidden representations, thereby quantifying the extent to which such information is linearly accessible within the model's latent space.

To assess both the accessibility and the complexity of the information encoded in the model representations, we employ three types of probes, covering a range of functional families from fully linear to boosted non-linear.

Since the hidden-state dimensionality exceeds the number of training instances on the $5\%$ subset, the linear and gradient-boosting probes operate on a shared preprocessing stack: standardization, removal of zero-variance components, and a principal-component projection to $\min(128, n_{\text{train}} - 1)$ components without whitening. The same preprocessing is adopted for the algorithm-selection probing of Section 3.4.

- **Linear probe:** a Ridge regressor ($\alpha = 10$) applied to the PCA-preprocessed representation. High performance with this probe indicates that the feature is linearly decodable via a penalized linear combination of the principal components of the hidden state.
- **Simple probe:** a one-hidden-layer MLP with $128$ hidden units, Rectified Linear Unit (ReLU) activation, dropout ($p = 0.2$), and weight decay ($10^{-4}$), trained with Adam and early stopping (patience $5$, restoring the best weights) on a $20\%$ internal validation split. It operates directly on the raw pooled hidden state, since its hidden layer provides a learned compression of the input.
- **Complex probe:** a LightGBM regressor ($200$ estimators, learning rate $0.05$, $\ell_2$ penalty $1.0$, minimum $20$ samples per leaf, feature subsampling $90\%$) applied to the same PCA-preprocessed representation. It is used to assess whether additional model capacity beyond the linear family yields substantial accuracy gains, which would suggest that the information is encoded in a non-linearly separable manner within the representation.

The evaluation protocol mirrors that adopted for direct querying (Section 3.2), comparing predicted and ground-truth feature values through exact-match and error-based metrics on the same validation instances. The comparison of

primary interest is between probing and direct querying, not between probe families; per-probe and per-pooling results are reported in C.

Notice that we interpret probing performance as evidence that a target property is decodable from the hidden representation under the chosen probe family. This should not be read as proof that the LLM explicitly computes the corresponding feature or internally implements the deterministic extraction procedure. In particular, probes may exploit feature-related correlations present in the instance distribution. For this reason, probing is used here as a complementary diagnostic of latent signal, not as a standalone claim of mechanistic understanding.

### 3.4 Algorithm Selection using Large Language Models

We further extend the probing analysis to examine whether the information encoded in the pooled layers of LLMs can support downstream tasks, specifically per-instance algorithm selection. In this setting, probing is formulated as a classification problem, where the objective is to identify, for each instance, the algorithm expected to achieve the best performance based on its latent representation.

The setup for this experiment mirrors that of the feature-probing analysis, employing the same probe types, with the key distinction that the target variable is now categoricalthat is, the label of the algorithm achieving the best performancerather than continuous. Each LLM-derived representation is paired with the ground-truth label of the best-performing algorithm for the corresponding instance, forming a dataset for supervised classification. A classifier probe is then trained to determine whether the identity of the winning algorithm can be decoded from the model hidden representations.

Performance is evaluated in terms of *set-aware accuracy*, which accounts for instances where multiple algorithms achieve identical best performance (ties). During training, one of the tied algorithms is randomly selected as the target label, whereas during evaluation, a prediction is considered correct if the classifier selects any of the algorithms attaining the best performance for that instance. Formally, given $n$ instances and for each instance $i$ a set of acceptable labels $S_i$, set-aware accuracy is defined as:

$$\mathrm{Acc}_{\text{set}} = \mathbb{E}_{(x_i, S_i)}\big[\, \mathbf{1}\{\hat{y}(x_i) \in S_i\} \,\big] \tag{1}$$

where $\hat{y}(x_i)$ denotes the predicted (single) label for instance $i$, described by the features $x_i$, and $\mathbf{1}\{\cdot\}$ is the indicator function. $\mathbb{E}$ is the expected value. A prediction is thus counted as correct if it matches any of the true *winning* algorithms for that instance.

To preserve balanced distributions across instances with different sets of winning algorithms, stratified sampling is performed over the powerset of possible labels (excluding the empty set). Each subset of algorithms defines a stratum, ensuring similar proportions of label sets in the training and evaluation splits.

To contextualize the results, we compare LLM-derived embeddings with three established baselines:

- **Most frequent winner:** a naive baseline that always predicts the algorithm most frequently achieving the best performance in the training set.
- **ISA-based classifiers:** classifiers trained on instance features derived from previous ISA studies.
- **Handcrafted feature classifiers:** classifiers trained on the handcrafted features used in the preceding experiments.

All representation families, namely LLM-derived embeddings, handcrafted features, and ISA-based features, are evaluated using the same downstream classifier families and the same data partitions. This protocol isolates the effect of the input representation from differences due to classifier capacity or train/test variability.

Classifier performance is evaluated under a stratified $k$-fold cross-validation protocol. The downstream classifiers considered in this setting are MostFrequentClassifier, LogisticRegression, MLPClassifier, and LightGBM, covering a naive baseline, linear decision models, and more flexible non-linear predictors. All classifiers are trained and tested on identical folds, ensuring paired comparisons across representations and models. The same partitions are then reused in the subsequent mixed linear analysis to assess whether LLM-derived representations achieve predictive power comparable to traditional ISA-based and handcrafted features.

The overall probing workflow is illustrated in Figure 3. Each instance is processed by the LLM, from which hidden representations are extracted and pooled to obtain fixed-length embeddings. These embeddings are paired with ground-truth labels to construct datasets for regression and classification probing. The regression probes test whether instance features can be decoded from the latent space, while the classification probes assess whether the same representations contain sufficient information to support downstream algorithm selection.

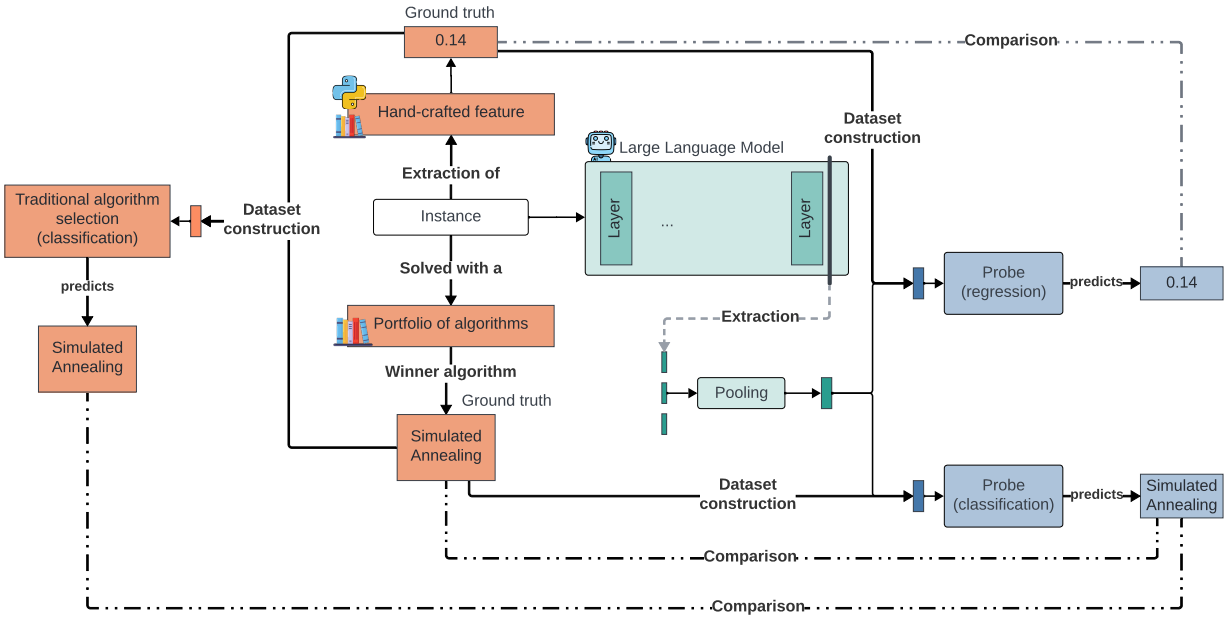



Figure 3: Overview of the probing process.

Table 4: Summary of the evaluated model pool and supported direct-querying modes. For gpt-oss models, the user-facing standard prompt is executed through native reasoning via Harmony.

| Model | Family | Parameters | Direct-querying modes |
|---|---|---|---|
| Llama-3.2-3B | Llama instruct | 3B | standard, Chain-of-Thought |
| Llama-3.1-8B | Llama instruct | 8B | standard, Chain-of-Thought |
| Llama-3.3-70B | Llama instruct | 70B | standard, Chain-of-Thought |
| gpt-oss20b | gpt-oss reasoning | 20B | native reasoning |
| gpt-oss120b | gpt-oss reasoning | 120B | native reasoning |

### 3.5 Experimental Setup

The experimental campaign is conducted on five open-weight LLMs spanning the 3B–120B parameter range: Llama-3.2-3B, Llama-3.1-8B, Llama-3.3-70B, gpt-oss20b, and gpt-oss120b. This model pool allows us to validate the robustness of the observed patterns across both instruct-style and reasoning-oriented open-weight families, while retaining full reproducibility of the experimental artifacts. Closed, commercial frontier models are not included for the reasons already discussed in Section 1.

All experiments are conducted on a stratified $5\%$ subset of the original benchmark collection. The subset is constructed independently for each problem by preserving the distribution of the downstream winner-label sets used in the algorithm-selection task and by maintaining coverage of the feature-complexity groups and instance representations used in the feature-extraction analyses. The resulting subset contains 440 BPP, 413 GCP, 223 JSP, and 265 KP instances, as reported in Table 3. All subsequent experiments use this same subset to ensure paired comparisons across querying, probing, and algorithm-selection analyses. Stratification is performed before model evaluation and is kept fixed across all models, prompting modes, pooling strategies, and downstream classifiers.

Figure 4 reports the token distributions for the four COPs on the stratified $5\%$ subset, distinguishing the three instance representations. Sequence length varies markedly across problems and formats: BPP and JSP remain relatively compact, KP is intermediate, and GCP is the longest and most variable case, especially in natural language. As expected, across all problems, natural-language inputs are consistently longer than code-like and standard representations. Nevertheless, all observed lengths remain within the context limits of the selected model pool, so no truncation is required.

The models are accessed via the Hugging Face interface and executed using the VLLM library, which provides improved robustness and efficiency over the standard transformers implementation. VLLM enables effective parallelization of both text generation in direct querying and hidden-state extraction in probing, allowing efficient processing of

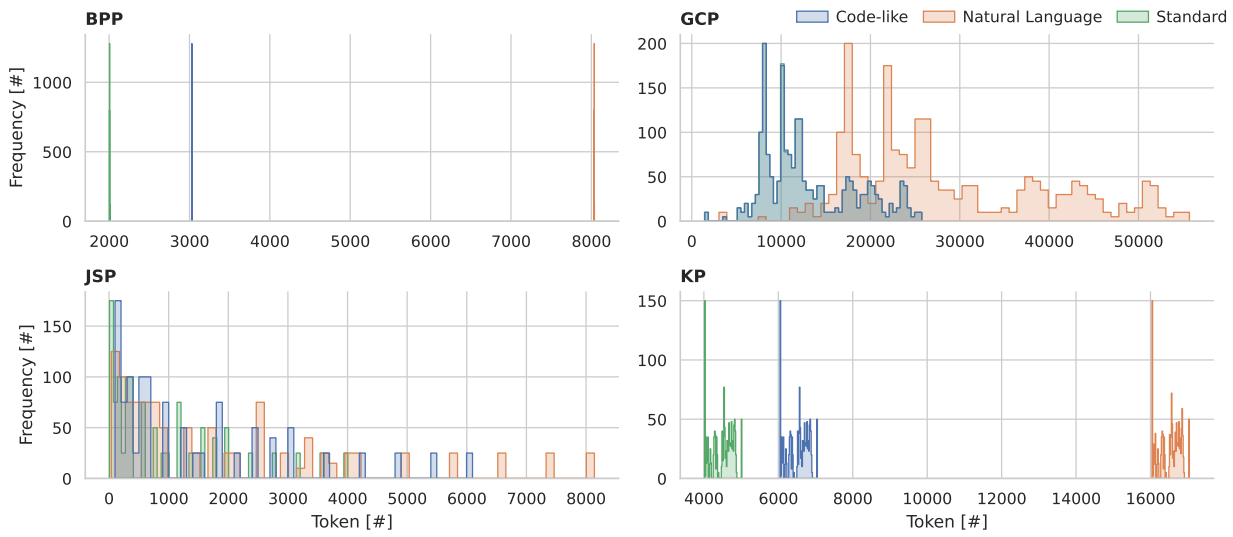



Figure 4: Number of tokens per problem and instance representation on the stratified 5% subset.

large batches and long input contexts. For gpt-oss models, direct querying is implemented through the Harmony message format, which supports native reasoning via a dedicated analysis channel while preserving the same feature-extraction task specification described in Section 3.2.

To ensure reproducibility, all experimental data and configurations are managed with dvc, while the source code is version-controlled through git. Experiments are executed on an NVIDIA DGX Station equipped with four NVIDIA A100 GPUs (80 GB each). Post-processing tasks, including regression and classification analyses, are carried out offline on a MacBook Pro (Apple M4, 32 GB RAM, 4+6 cores). Random seeds, problem sets, and replicate blocks are fixed across all runs to ensure consistent experimental conditions. The complete git repository, including configuration files, experimental scripts, and processing pipelines, will be released as an open repository upon paper acceptance.

The probing models are implemented using `scikit-learn`, with `LinearRegression` for regression, `MostFrequentClassifier` and `LogisticRegression` for classification, and `pytorch` with the `skorch` adapter for the MLP probes applied to both tasks. Furthermore, LightGBM is used for both regression and classification. The `scikit-learn` framework was selected for its interoperability and consistency, allowing a unified experimental pipeline across all probe types. An ad hoc adapter class was developed for the set-aware classification task to ensure the correct computation of its evaluation metric within the same workflow.

Regarding the algorithm portfolios, the results are drawn from the original ISA studies (also referenced in Table 3), together with the corresponding instances and features labeled as ISA features. The ground-truth features employed in the experiments presented in Sections 3.2 and 3.3 are computed by dedicated feature-extraction scripts developed in Python. The handcrafted feature extractors are included in the project code base and integrated within a data preparation pipeline that processes the benchmark instances and computes the relevant features for the downstream LLM-related experiments.

The experiments are organized as follows. In the direct-querying setting, each problem instance in the stratified subset is paired with each target feature and queried under the selected prompt variant and execution mode. In the probing setting, only the instance representation is provided to the network; the last hidden state is extracted, pooled, and used to train regression probes on a train–validation split. In the algorithm-selection setting, both pooled hidden states and manually designed features (handcrafted or ISA-based) are used as classifier inputs under a stratified $k$-fold cross-validation protocol. This design yields a large but computationally tractable number of LLM runs and downstream learning experiments, while preserving consistent comparisons across models, prompting choices, and downstream baselines.

## 4 Experimental Analysis

This section reports the results of the experimental analysis conducted to evaluate the behavior and representational properties of LLMs when applied to COPs. The experiments follow the methodology introduced in Section 3, en-

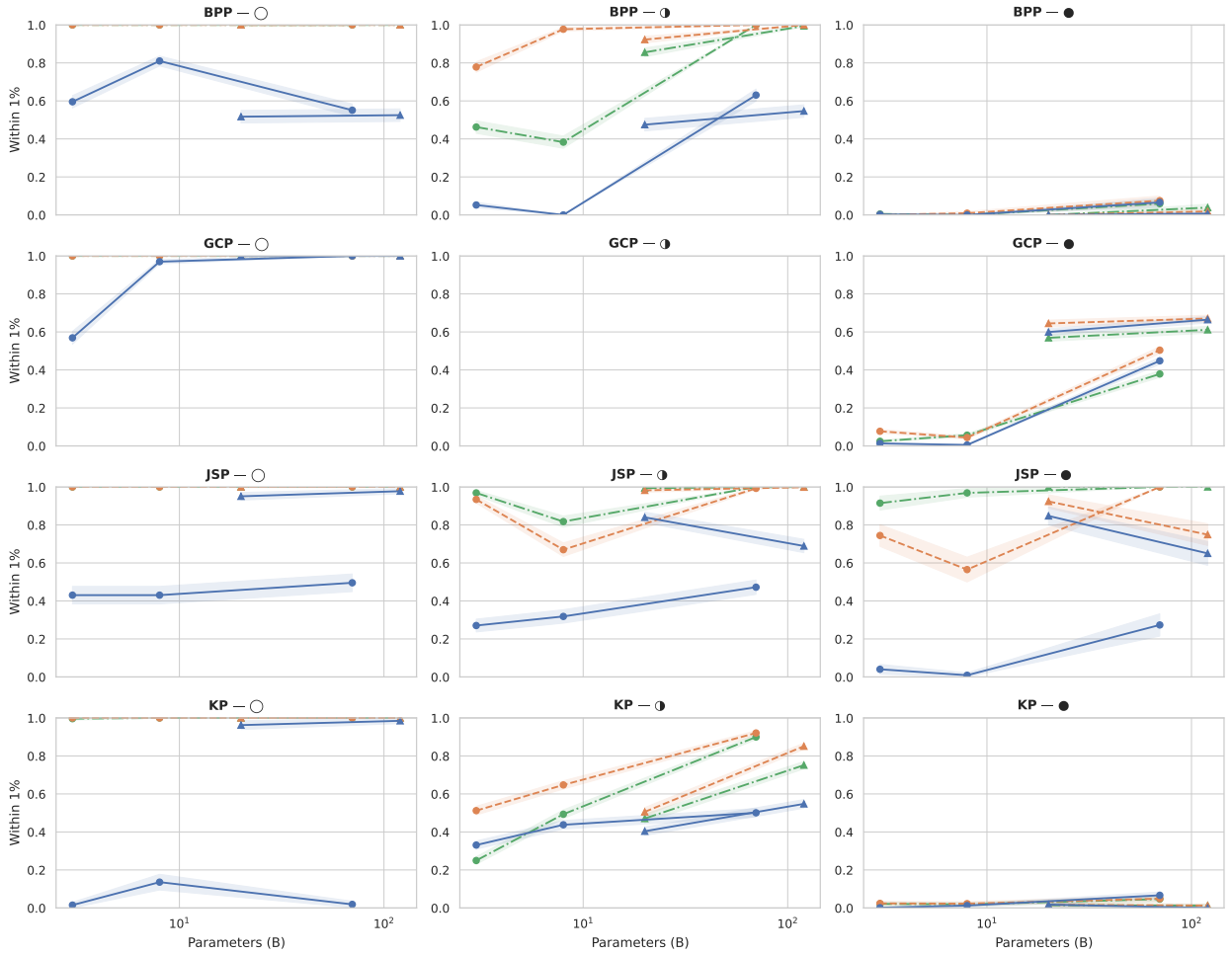



Figure 5: Direct-querying performance as model size increases, measured as the proportion of predictions within 1% of the ground truth. Each panel corresponds to a feature-complexity group. Results correspond to standard direct querying for Llama models and native reasoning for gpt-oss models.

compassing three complementary analyses: direct querying, feature probing, and algorithm selection probing. Each analysis focuses on a distinct aspect of model behavior: respectively, explicit feature identification, implicit feature encoding, and predictive modeling of algorithmic performance.

### 4.1 Direct Querying of Problem Features

We begin the experimental analysis by examining the behavior of LLMs in the direct querying setting, where models are explicitly prompted to extract numerical features from problem instances. Performance is assessed using multiple metrics: Mean Absolute Error (MAE), computed as the average absolute deviation from the ground truth; the proportion of exact matches (EQUALS); and the proportions of predictions within 1% (WITHIN 1%) and 5% (WITHIN 5%) deviation from the ground truth values. These latter metrics provide a more sensitive view of near-accurate predictions, complementing the MAE analysis. In the following, we focus primarily on the WITHIN 1% metric for clarity. Robustness checks on MAE, exact-match accuracy and NULL RATE are reported in B, and the full set of metrics is available in the companion repository.

#### 4.1.1 Effect of feature complexity and model scale

Figure 5 shows direct-querying performance as a function of model size across feature-complexity groups. Across all problems, feature complexity is the dominant factor: ○ features are recovered reliably, while performance degrades substantially for ◓ and ● features.

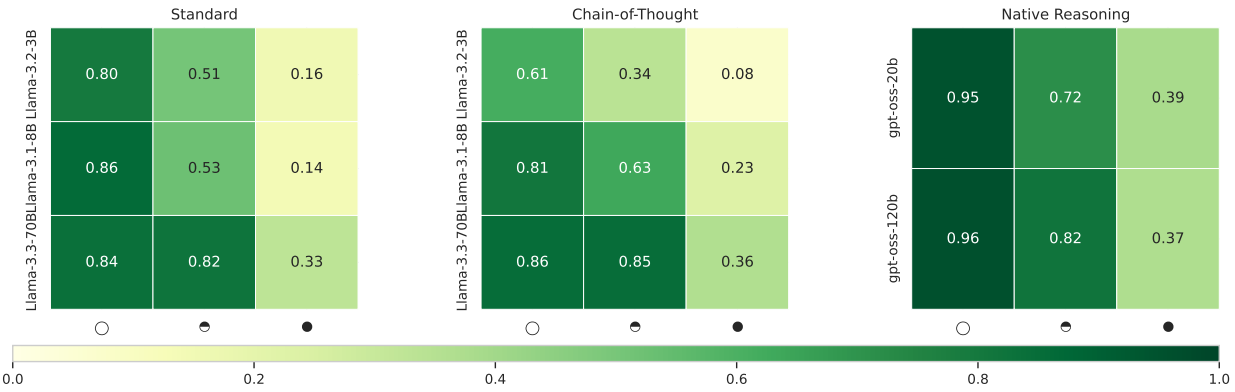



Figure 6: Effect of Chain-of-Thought prompting on direct-querying performance, shown as the change in mean Within 1% accuracy relative to standard direct querying. Results are shown across model sizes, reasoning modes, and feature-complexity groups.

Model scaling has a non-uniform impact. For ○ features, increasing model size yields only marginal improvements, as performance is already near saturation. For ◓ features, scaling leads to clear gains, particularly for natural-language and code-like representations. For ● features, improvements remain limited, indicating that increased capacity alone does not fully compensate for the need for structured computation.

The same scaling pattern emerges when the analysis is repeated with alternative response variables capturing prediction error, namely normalized MAE and exact-match accuracy: normalized MAE slopes are negative across all non-degenerate cells, indicating that the average error shrinks by approximately $40$–$60\%$ for each parameter doubling on the ◓ and ● groups, and exact-match slopes are positive with comparable goodness of fit. Per-metric regression coefficients are reported in B (Table 5 on page 38).

### 4.1.2 Effect of representation

Representation plays a systematic but non-uniform role. The standard representation is frequently the least effective, while natural-language and code-like inputs generally yield higher accuracy. However, their relative ordering is not consistent across problems and feature-complexity groups.

In particular, while natural-language representations often provide the strongest performance for ○ features, code-like representations can match or outperform them in other settings, especially as feature complexity increases. This suggests that LLMs benefit from representations that expose semantic structure or explicit relational patterns, rather than from rigid encodings such as the standard representation.

### 4.1.3 Effect of prompting and reasoning

Figure 6 provides a detailed view of the impact of CoT prompting on direct-querying performance across model scales and feature-complexity groups.

The effect of CoT is strongly scale-dependent and non-monotonic. For the smallest model (3B), CoT consistently degrades performance across all feature types, suggesting that explicit reasoning introduces additional overhead without improving the model's ability to extract the correct value. In this regime, the model appears unable to effectively leverage intermediate reasoning steps.

For intermediate-scale models (8B), the effect becomes selective. CoT provides improvements primarily for ◓ and ● features, where some degree of decomposition is beneficial, while offering little or no advantage for ○ features. This indicates that CoT can act as a useful scaffold for moderately complex reasoning, but only when the underlying model capacity is sufficient.

For larger models (70B), CoT consistently improves performance across all feature-complexity groups. At this scale, explicit reasoning no longer acts as a burden, but instead helps organize the computation, leading to more reliable feature extraction.

These results suggest that CoT does not have a uniform effect, but interacts critically with model scale. Below a certain capacity threshold, it can hinder performance; at intermediate scales, it provides targeted benefits; and at larger scales,

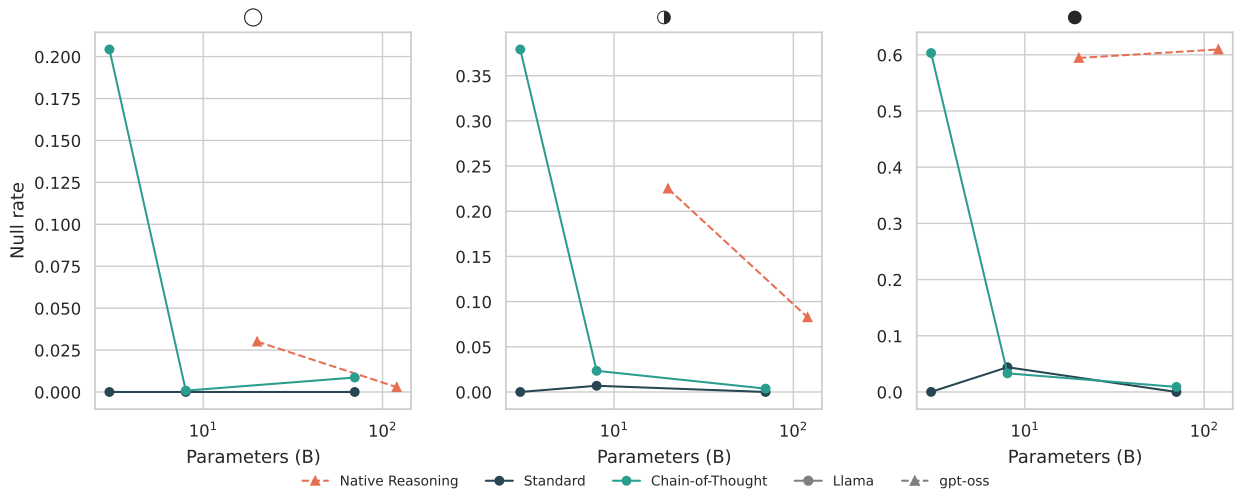



Figure 7: Null prediction rate as a function of model size. Higher values indicate increased abstention.

it becomes broadly advantageous. Equivalent heatmaps computed on MAE and exact-match accuracy show the same scale-dependent shape and are made available in the companion repository.

Notably, these effects interact with the models' failure modes, as discussed below, where reasoning-oriented configurations also differ in their tendency to abstain.

#### 4.1.4 Failure modes: abstention vs incorrect outputs

A qualitative difference emerges between model families. While Llama models consistently produce numeric outputs across all configurations, gpt-oss models exhibit a different behavior, particularly for complex features. As shown in Figure 7, gpt-oss models frequently abstain from producing a value for ● features, with null rates approaching saturation in some cases. This contrasts with Llama models, which tend to produce outputs even when incorrect. As a result, improvements in apparent accuracy must be interpreted jointly with the corresponding increase in abstention.

For this reason, accuracy metrics for reasoning-oriented models should be interpreted jointly with coverage. A model that abstains more frequently may avoid incorrect numerical commitments, but it also reduces the fraction of instances for which direct feature extraction produces a usable value.

This abstention behavior also varies systematically with model size: regressing NULL RATE on $\log_2(\text{model size})$ yields a slope close to zero for ○ and ◓ features, but becomes the largest of the three considered metrics at the ● level (around $+0.07$ per parameter doubling, with $R^2 \approx 0.25$ across the three representations; see B). Larger models in the gpt-oss family therefore do not principally improve their accuracy on high-computation features; rather, they increase the share of inputs on which they decline to commit to a value.

#### 4.1.5 Computational cost

Figure 8 reports the average generation time per query as model size increases across problems and feature-complexity groups, and Table 6 (in B) summarizes the corresponding mean times by model and mode. As expected, computational cost grows substantially with model size: average generation time spans roughly 0.2–0.5 s on the 3B model, 0.4–1.2 s on the 8B model, and 1.2–3.5 s on the 70B model. An interesting exception is gpt-oss-120b, which averages 0.4–0.7 s despite its parameter count: its mixture-of-experts architecture activates only a fraction of the weights per token, making it faster than the dense Llama-3.3-70B at all complexity levels.

These figures should be read in light of the cost of the handcrafted alternative. The features used as ground truth in this study are computed by simple deterministic procedures over instance arrays of $10^2$–$10^4$ elements, with extraction-complexity features being $O(1)$ lookups, low-computation features being $O(n)$ scans for minimum or maximum, and high-computation features being $O(n)$ averages or simple aggregations. On the same hardware used to host the inference workload, all these procedures execute in tens of microseconds to a few milliseconds per instance. The LLM extraction therefore takes between three and five orders of magnitude longer than the handcrafted alternative for the same output, before accounting for GPU energy consumption, which the handcrafted procedures do not incur at all.

These cost increases are not always matched by proportional gains in predictive performance. For ○ features, where accuracy is already near saturation, larger models incur substantially higher inference costs without meaningful improvements. For ◓ and ● features, scaling yields measurable gains, but at a steep computational expense. The practical implication is that, when handcrafted extractors exist and are reliable, using LLMs as feature replacement is hard to justify on cost grounds; their value lies elsewhere, namely in settings where instances are described in formats for which no handcrafted extractor is available, or as latent feature providers as discussed in Section 4.3.

A similar trade-off emerges when considering explicit reasoning. Figure 9 compares the average generation time of standard direct querying and CoT prompting for Llama models on BPP (the corresponding results for the other problems show the same qualitative pattern). As expected, CoT prompting increases generation time across model sizes and representations, but the magnitude of this increase is not uniform. The aggregate CoT-to-standard time ratios are $1.87\times$ on the 3B model, $1.27\times$ on the 8B model, and $1.53\times$ on the 70B model. For the 3B model, the additional cost is most pronounced on the more complex feature groups, where explicit reasoning introduces a substantial overhead without delivering corresponding gains in performance. For the 8B model, the increase in generation time is more limited, while the accuracy gains remain selective. For the 70B model, the strongest time increase is observed for the standard representation, which is also the least effective one overall; in this case, CoT yields only modest improvements, indicating that additional reasoning effort does not necessarily translate into proportional gains. The smallest model therefore pays the largest relative cost for CoT prompting, while also being the only one for which prompting consistently degrades accuracy. Taken together, these results show that both model scaling and explicit reasoning improve direct querying only under specific conditions, while always increasing computational cost. This makes the trade-off between performance and efficiency central to the evaluation of LLM-based feature extraction.

We do not report direct energy measurements, and therefore do not claim a complete energy-efficiency analysis. Nevertheless, the observed inference times and hardware requirements already indicate a substantially larger resource footprint than deterministic CPU-based feature extraction. The cost comparison should therefore be interpreted as conservative: LLM-based direct extraction is slower by several orders of magnitude and also requires substantially more specialized hardware.

#### 4.1.6 Summary

Overall, direct querying is effective for extracting explicit features, but remains fragile for features requiring structured computation. Model scale, input representation, and explicit reasoning all influence performance, yet their effects are strongly mediated by feature complexity. In particular, CoT prompting does not have a uniform effect: it can be detrimental at small scale, selectively beneficial at intermediate scale, and more consistently helpful only for larger models, always at an additional computational cost. Moreover, model families differ not only in accuracy, but also in how they handle uncertainty, with reasoning-oriented models more frequently favoring abstention over potentially incorrect outputs.

To investigate whether these limitations arise from the models' reasoning behavior or from the nature of their internal representations, the next analysis turns to feature probing.

### 4.2 Feature Probing

We now turn to probing to investigate whether feature-related information is encoded in the internal representations of LLMs. In contrast to direct querying, which evaluates the model's ability to explicitly retrieve a feature value, probing assesses whether the same information can be recovered from hidden-layer activations using simple supervised models.

In this setting, each instance is processed by the LLM without an explicit feature query, and the resulting hidden states are used as input to regression probes trained to predict the corresponding feature values. This allows us to measure the *latent decodability* of feature information, independently of the model's ability to generate the correct value through its output layer.

Direct querying and probing capture two distinct aspects of model behavior. Direct querying evaluates *explicit recoverability*, i.e., whether the model can generate the correct feature value when prompted. Probing, instead, evaluates *latent decodability*, i.e., whether the same information is encoded in the model's internal representations and can be extracted by a downstream model. These two properties do not necessarily coincide.

Figure 10 compares direct-querying and probing performance across feature-complexity groups, while Figure 11 provides a joint view of the two signals.

The aggregate comparison shows that the relationship between the two signals depends on feature complexity and representation. In some configurations, probing achieves higher accuracy, particularly for more complex features,

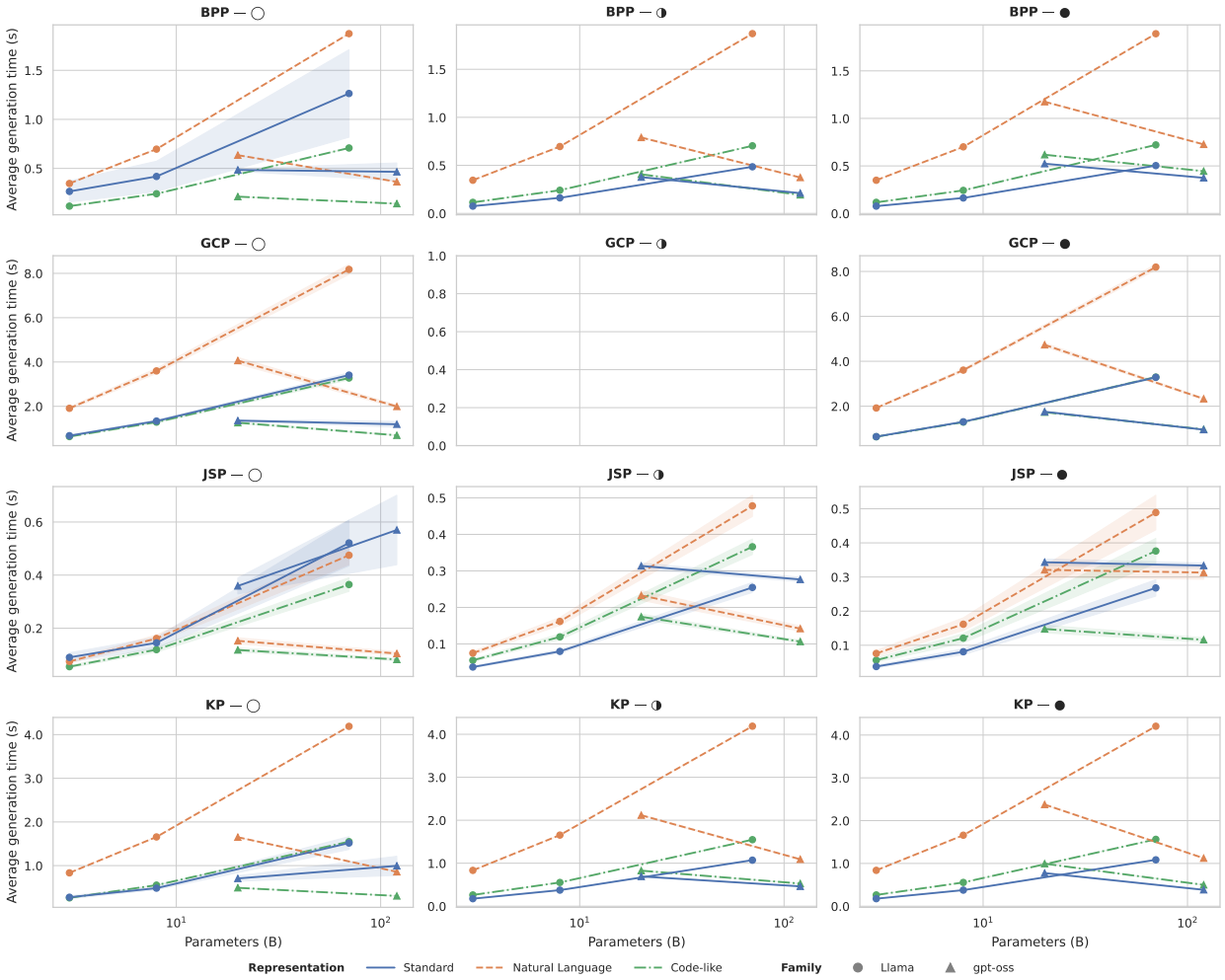



Figure 8: Direct-querying computational cost as model size increases, measured as average generation time per query. Each panel corresponds to a feature-complexity group. Results correspond to standard direct querying for Llama models and native reasoning for gpt-oss models.

whereas in others direct querying remains competitive or superior. This indicates that neither method universally dominates the other.

The joint view in Figure 11 makes this complementarity more explicit. If direct querying and probing were measuring the same property, most points would concentrate near the diagonal. Instead, the scatter shows a substantial dispersion, with many cases deviating markedly from the line of equality. This indicates that high latent decodability does not necessarily imply high explicit recoverability, and vice versa.

The off-diagonal regions of Figure 11 are particularly informative. Points with high probing accuracy and low querying accuracy indicate features whose signal is available to a downstream decoder but not reliably exposed through generation. Conversely, points with high querying accuracy and lower probing accuracy indicate cases where the model can follow the prompt or exploit explicit input structure without producing a representation that is equally useful to the selected probe. The two measurements therefore diagnose different failure modes.

The marginal distributions reinforce this interpretation by showing that the two signals have different global profiles. Direct querying tends to be more polarized, with many cases concentrated near either high or low explicit recovery, whereas probing exhibits a broader spread, suggesting that feature information may be present in the representations even when the model's output fails to retrieve it reliably.

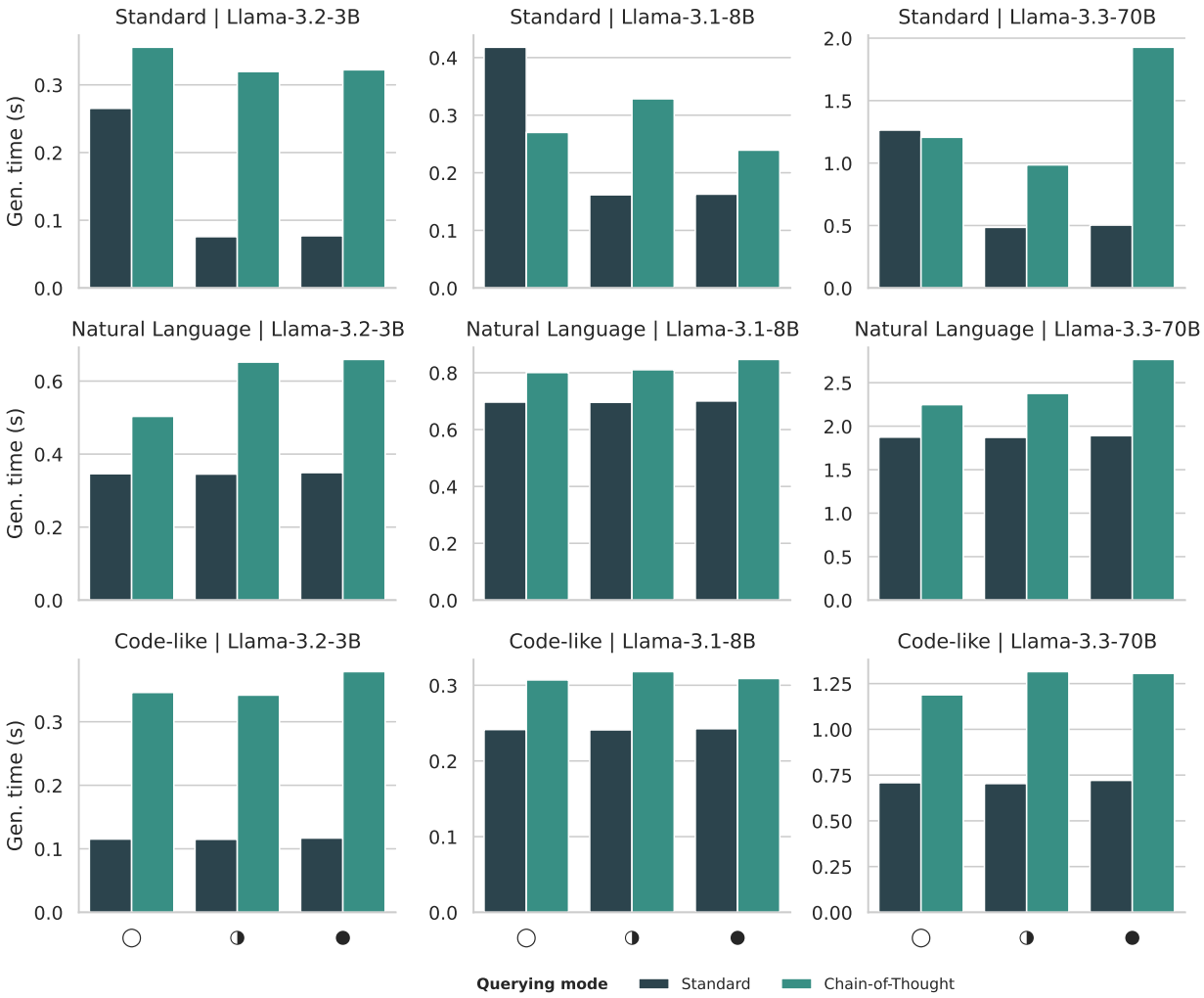



Figure 9: Comparison of average computational cost (generation time) between standard direct querying and CoT prompting for Llama models on the BPP problem. Each panel corresponds to a representation–model combination.

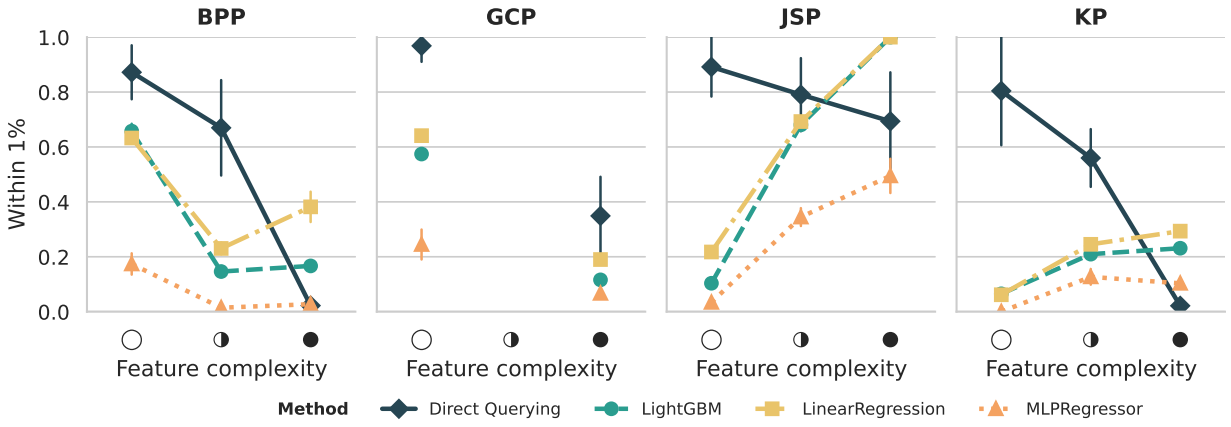



Figure 10: Comparison between direct querying and probing performance, measured as Within 1% accuracy across feature-complexity groups.

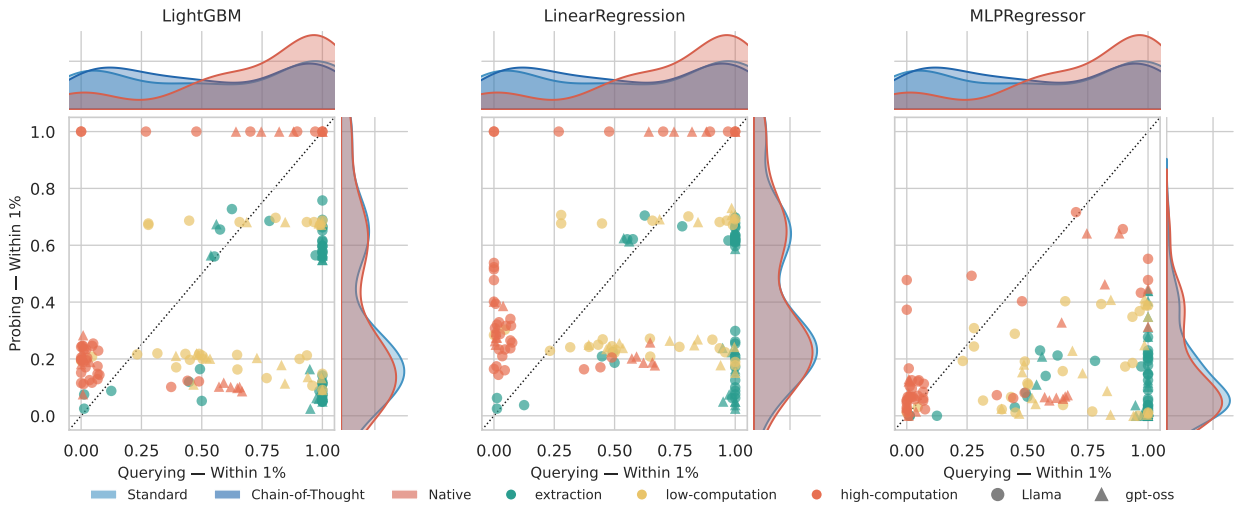



Figure 11: Joint comparison of direct-querying and probing performance in terms of Within 1% accuracy. Marginal distributions summarize the behavior of the two signals.

The present analysis is conducted on the stratified 5% subset introduced in Section 3.5, which limits the statistical strength of the observed patterns and introduces some variability across configurations. The qualitative complementarity between direct querying and probing is nevertheless evident across all five evaluated models.

Taken together, these results show that direct querying and probing provide complementary but non-equivalent signals about feature information in LLMs. Direct querying measures explicit recoverability, whereas probing captures latent decodability. The broader implications of this divergence are discussed in Section 5.

This distinction has important implications. It suggests that part of the limitation observed in direct querying arises from the generative process itself, rather than from the absence of relevant information in the model. At the same time, the variability of probing results indicates that not all feature information is robustly encoded in a form that can be reliably exploited.

These findings motivate the next step of the analysis. If latent representations contain information that is not always accessible through direct querying, a natural question is whether such information can nevertheless support downstream tasks. In the following section, we evaluate this hypothesis through an algorithm selection setting, assessing whether LLM-derived representations can act as effective instance descriptors.

### 4.3 Algorithm Selection Probing

We now assess whether this latent information can be exploited in a downstream algorithm-selection task. Specifically, we test whether LLM-derived embeddings can match the effectiveness of manually designed features in predicting the best-performing algorithm for each instance.

To address this question, we assess whether LLM-derived embeddings can match the effectiveness of manually designed features in the key downstream task of algorithm selection. A complete summary of the statistical analyses performed below is reported in D.

This analysis is also conducted across all five open-weight models considered in the study, spanning the 3B–120B range and covering both the Llama instruct and gpt-oss reasoning families. This enables us to evaluate whether the predictive utility of LLM-derived representations is a stable property across models, rather than a phenomenon specific to a single configuration.

#### 4.3.1 Comparative Statistical Analysis of Classifier Performance

This subsection presents a comparative statistical analysis of the classifiers employed in the algorithm selection task, aiming to determine whether their predictive performances differ significantly. The analysis serves as a preliminary validation step to ensure that the choice of classifier does not confound the interpretation of subsequent results. To this end, multiple classifiers are evaluated under identical data partitions.

To assess the influence of classifier type on prediction accuracy,[9] we fitted a linear mixed-effects model with block (*problem*, *replicate*) as a random intercept, including both *classifier* and *model* as fixed factors. This formulation allows us to evaluate classifier effects while controlling for the specific representation source used in the downstream task.

The overall effect of classifier was highly significant (likelihood-ratio test, $LR = 742.403$, $df = 3$, $p = 1.34\times10^{-160}$), indicating that classifier choice substantially affects predictive accuracy. Relative to the MostFrequentClassifier baseline, LightGBM achieved the strongest improvement ($+0.041$, $p < 0.001$), followed by LogisticRegression ($+0.014$, $p < 0.001$), whereas the MLPClassifier showed a statistically significant decrease ($-0.013$, $p < 0.001$).

Controlling for classifier, all evaluated representation sources yielded positive coefficients of similar magnitude relative to the baseline condition, with estimated effects in the range of approximately $+0.016$ to $+0.019$. This indicates that the choice of downstream classifier remains a stronger determinant of performance than the specific model representation among those considered here. The estimated random-intercept variance at the (problem, replicate) block level ($\sigma^2 = 0.032$) further suggests substantial problem-dependent variability, but does not alter the overall ranking of classifiers.

Figure 12 shows the distribution of classifier accuracies across problems and models. The figure confirms that the relative ranking of classifiers is stable across both problem types and underlying model representations: LightGBM consistently provides the strongest predictive performance, LogisticRegression yields a smaller but robust improvement over the MostFrequentClassifier baseline, and MLPClassifier underperforms relative to the other classifiers.

Performance differences are also problem-dependent. In particular, KP is consistently the most challenging problem, with substantially lower accuracies across all classifiers and models. By contrast, JSP yields the highest overall accuracies, despite involving a larger number of candidate algorithms, indicating that problem difficulty in this setting is not determined simply by the number of classes but also by the structure of the task and the set-aware evaluation criterion. BPP and GCP stay at an intermediate but relatively stable range. This pattern aligns with the non-zero between-problem variance estimated by the mixed-effects model.

Given these results, the subsequent analyses focus on LightGBM and LogisticRegression, which provide the most stable and competitive predictive behavior. This choice ensures that downstream comparisons primarily reflect differences in representational quality rather than limitations of the classification models themselves.

#### 4.3.2 Comparative Analysis of ISA-Based and Handcrafted Feature Sets

We now compare the predictive behavior of the ISA-based and handcrafted feature sets to assess whether they yield systematically different outcomes in the algorithm selection task. While ISA-based features are designed as comprehensive descriptors and may include or extend the handcrafted ones, it is nevertheless important to evaluate whether this broader coverage translates into superior predictive power. Establishing this comparison also provides a consistent baseline for the subsequent analysis, where the predictive power and robustness of manually engineered features are contrasted with those derived from LLM hidden representations.

To examine whether the predictive behavior of the two baseline feature sets differs systematically, we fit a linear mixed-effects model with *classifier* as a fixed factor and *block*, defined as the (*problem*, *replicate*) pair, as a random intercept.

The analysis did not identify a significant main effect of the baseline representation (LRT $\chi^2(1) = 0.819$, $p = 0.365$), indicating that ISA-based and handcrafted features yield comparable predictive performance in this setting. The estimated coefficient for the ISA representation ($+0.007$, 95% CI $[-0.009, 0.023]$) is small and not statistically significant.

The classifier effect is consistent with the previous analysis, with LightGBM outperforming Logistic Regression ($+0.024$, $p = 0.004$).

Complementary tests reinforce this interpretation. A Tukey HSD comparison does not detect significant differences between ISA-based and handcrafted features, while a Wilcoxon-based Two One-Sided Test (TOST) fails to establish practical equivalence within a margin of $\varepsilon = 0.01$. However, the low statistical power of the equivalence test suggests that these results should be interpreted cautiously.

Overall, these findings indicate that ISA-based and handcrafted features exhibit similar predictive performance, with no robust evidence of a systematic advantage for either representation. This makes them suitable and comparable baselines for evaluating LLM-derived representations in the subsequent analysis.

[9] For these analyses the term accuracy has to be intended in the set-aware sense presented in Section 3.4




Figure 12: Classification accuracy by problem, classifier, and model. The relative ranking of classifiers remains stable across models, confirming that classifier choice is a primary driver of performance.

#### 4.3.3 Analysis of Representation and Pooling Effects on Probing Performance

To investigate how different combinations of instance representation and pooling strategy influence probing performance, a full-factorial analysis is conducted to evaluate main and interaction effects between the three representations (standard, code-like, and natural language) and the three pooling strategies (*mean*, *max*, and *last*) on classifier accuracy. The analysis controls for problem-specific variability to isolate systematic effects attributable to representation and pooling choices, providing insight into how these factors shape the extractable information from LLM hidden activations.

We fitted a linear mixed-effects model including *classifier* as a fixed factor and *block* (i.e., the pair (*problem*, *replicate*)) as a random intercept to assess how different combinations of representation and pooling activations influence classifier performance in the algorithm selection task. The model evaluated the main and interaction effects of the two factors, while accounting for systematic differences between classifiers. In addition, the effect of the underlying LLM model was included as a fixed factor. The likelihood-ratio test indicates that the model effect is not significant ($\chi^2(4) = 6.381$, $p = 0.172$), suggesting that differences across models do not substantially affect downstream performance in this setting.

Likelihood ratio tests indicate that the effect of pooling is significant ($\chi^2(6) = 37.389$, $p = 1.48 \times 10^{-6}$), whereas the effect of representation is not statistically significant ($\chi^2(6) = 11.353$, $p = 0.078$). A main effect of classifier was also observed, with LightGBM outperforming Logistic Regression by approximately $0.027$ accuracy points ($p < 0.001$). Overall, these results indicate that classifier choice dominates downstream performance, while pooling has only a modest effect and neither representation nor LLM model is significant. No systematic representation–pooling interaction was detected.

The estimated coefficients indicate only minor differences between representations, with both natural language and code-like showing small, non-significant deviations from the standard baseline. This is consistent with the likelihood-ratio test, which does not support a statistically significant effect of representation. Overall, these results suggest that the choice of representation has limited influence on downstream performance in the probing setting.

Among pooling strategies, a statistically significant effect is observed overall. In particular, *last* pooling shows a small but significant decrease in performance relative to the *mean* baseline ($-0.010$, $p = 0.002$), whereas *max* pooling does not exhibit a significant difference. Although these effects are statistically detectable, their magnitude remains small.

Figure 13 illustrates the mean accuracies (with 95% confidence intervals) obtained for each problem under different combinations of representation and pooling strategy. Across all problems, *max* pooling consistently provides a slight advantage over the other two pooling methods, regardless of the underlying representation. The effect of representation is minimal and does not show a consistent pattern across problems, whereas pooling exhibits small but more systematic differences.

Overall, representation choice has no significant impact on downstream performance, while pooling introduces only minor effects. These results suggest that the predictive signal encoded in LLM representations is largely robust to variations in representation, pooling strategy, and model scale.

For the final analysis, we adopt a single configuration consisting of *max* pooling and the standard representation to provide a consistent and reproducible basis for comparison with baseline feature sets.

#### 4.3.4 Comparative Analysis of ISA and LLM-Derived Representations for Algorithm Selection

We finally compare ISA-based features with LLM-derived representations to assess whether latent embeddings can support competitive downstream performance when paired with the best-performing downstream classifier, namely LightGBM.

The comparison does not support a uniform advantage of the ISA baseline over LLM-derived representations. Instead, the observed differences are small and problem-dependent. In particular, the only problem showing a statistically significant difference is BPP, where LLM-derived representations achieve slightly higher accuracy than the ISA baseline ($+0.041$ accuracy points, $p = 0.001$). No robust evidence of systematic differences emerges for the other benchmark problems.

To further inspect this pattern, we analyze the cross-model accuracies reported in Figure 14. The figure shows that LLM-derived representations achieve predictive performance that is broadly comparable to the ISA baseline across models, with relatively small deviations in absolute accuracy. Even in the case of BPP, where the global test detects a significant difference, the post-hoc comparisons among the LLM-based configurations do not reveal strong separation, suggesting that the practical gap remains limited.

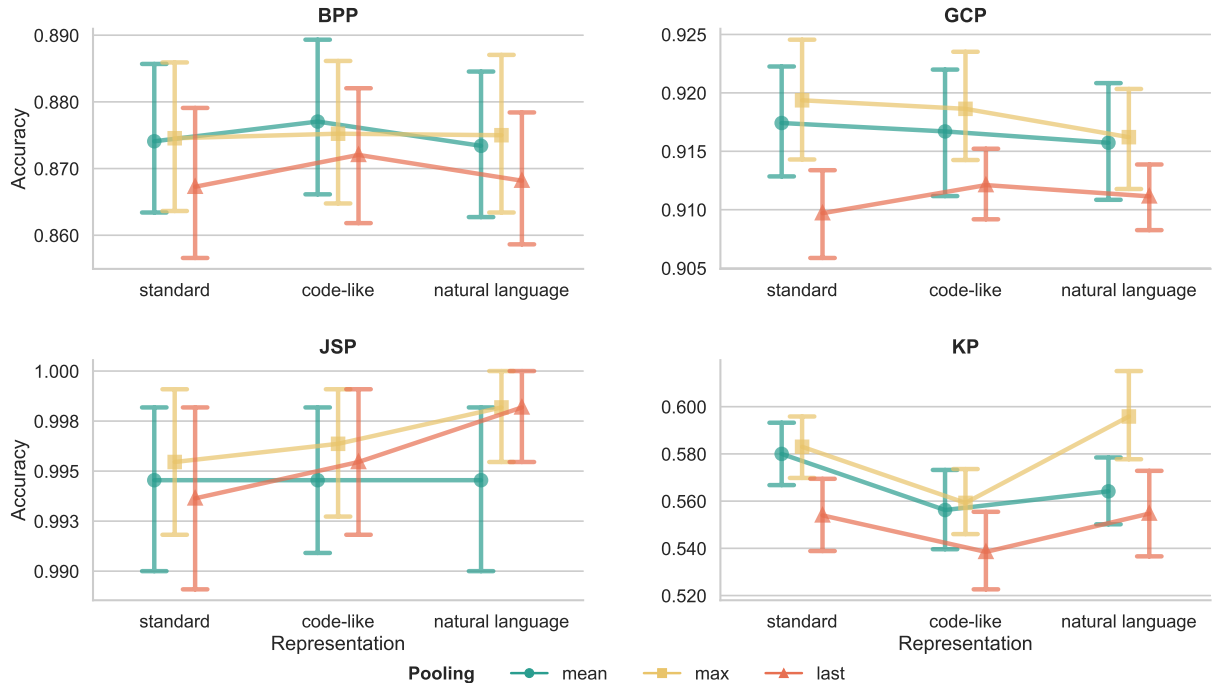



Figure 13: Effects of representation and pooling in the full factorial experiment. Each panel shows the mean accuracy (95% CI) for combinations of representation and pooling, averaged across problems and classifiers.

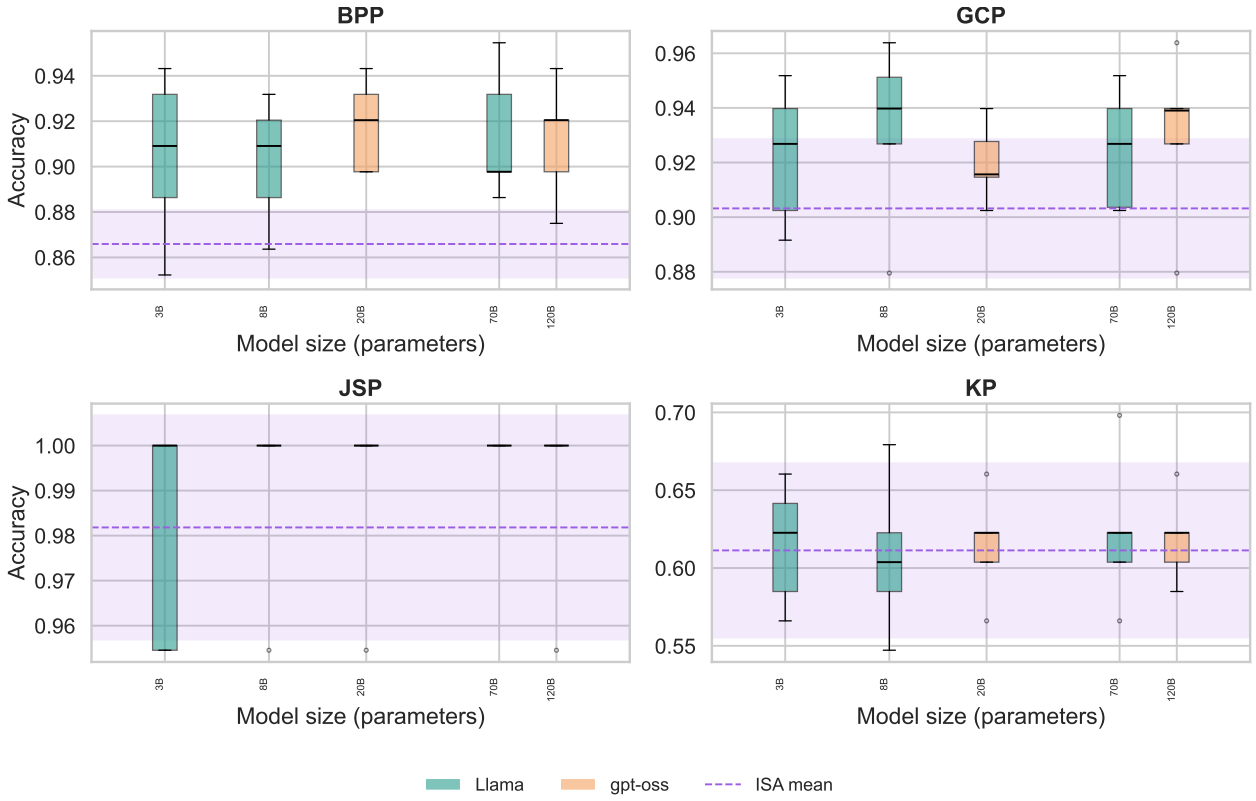



Figure 14: Comparison of algorithm selection accuracy across problems between the ISA baseline and LLM-derived representations, using LightGBM as the downstream classifier. Results are reported across the evaluated models. The dashed horizontal line denotes the mean ISA accuracy, and the highlighted band indicates the interval within one standard deviation of the ISA baseline. Overall, performance differences remain small and problem-dependent.

## 5 Discussion: Implicit Signal vs. Explicit Retrieval

The experimental analysis in Section 4 reveals a recurrent functional asymmetry between what LLMs produce as explicit numerical answers and what their internal representations encode. Direct querying recovers feature values reliably only when they can be read off the input with little or no computation, while probing decodes the same features from hidden states with consistently lower error, particularly on the more demanding feature-complexity groups (Figures 10 and 11). We collect in this section the elements of this dissociation under a common interpretive frame and discuss what they imply for the use of LLMs as feature providers in combinatorial-optimization pipelines. Our reading remains descriptive: the data support a clear separation between *explicit recoverability* and *latent decodability*, but we do not attempt to settle the broader question of whether such separation should be read as evidence of comprehension or its absence.

We deliberately avoid interpreting this dissociation as direct evidence either for or against model comprehension. The experiments show that feature-related information can be decodable from hidden states while remaining unavailable to reliable explicit generation. Whether this reflects partial understanding, statistical association, or a limitation of the decoding and reasoning process is a broader mechanistic question beyond the scope of this study.

**Failure modes** The dissociation is reinforced by a second observation: models in our pool do not differ only in how often they retrieve the correct value, but in *how* they fail when retrieval is unsuccessful. Three failure modes emerge from Figure 7 and from the per-cell breakdown in B, each associated with a specific portion of the model pool. The first is *calibrated abstention*, observed primarily in gpt-oss models on ● features: when reliable computation is not possible, the model returns `null` under structured decoding, with abstention rates approaching saturation in some configurations. The second is *over-confident hallucination*, observed primarily in Llama models and most pronounced under CoT prompting at intermediate scale: the model commits to a type-valid but incorrect numeric value rather than abstaining, a behavior that structured decoding does not prevent because the constraint is on output type, not on output validity. The third is *structured-output failure*, observed in the smallest model (Llama-3.2-3B) under CoT prompting on ● features. Producing an explicit reasoning trace consumes most of the available generation budget and leaves the model unable to emit a well-formed value field on a substantial share of features; the structured decoder then records the result as null. The output is identical in form to a calibrated abstention, but does not carry the same meaning: it reflects a failure to complete the required output structure rather than a decision to refuse. The three modes are not equivalent for downstream use: calibrated abstention carries usable information about the model's own limits, over-confident hallucination silently corrupts the output, and structured-output failure leaves the pipeline with non-informative gaps.

**Latent encoding without exact computation** A consistent picture emerges when the dissociation between querying and probing is combined with the scaling regressions reported in B. Probing shows that feature-relevant information is encoded in the model's hidden states: a regressor trained on these states recovers feature values from instances that the model itself fails to extract under direct querying, and downstream classifiers built on the same representations match the predictive power of manually engineered features in algorithm selection (Section 4.3). At the same time, explicit retrieval improves with scale but does not match handcrafted extraction: normalized MAE decreases by roughly $40\%$ to $60\%$ per parameter doubling on the ◓ and ● groups, while the null-rate slope on ● features grows in parallel ($+0.07$ per parameter doubling, $R^2 \approx 0.25$). Within the 3B–120B range we evaluate, larger models compress this residual error and become, at the same time, more likely to abstain on features where the answer requires actual computation. This is consistent with a reading in which the latent representation captures statistical regularities of the input–feature relation without instantiating a procedure that computes the feature exactly. We summarize this distinction as one between an *encoded statistical signal* and an *executable computation*. The probing pipeline recovers the former with comparable accuracy across model sizes; the latter is the deterministic procedure that maps the raw instance to the feature value, and the model's output only approximates it.

**Where latent representations contribute** The implications of the dissociation for downstream tasks are clearest in the algorithm-selection setting, where hidden-state representations match the predictive performance of handcrafted feature sets across all five evaluated models (Section 4.3), with no robust evidence of advantage for either side. Given this parity, the two approaches are most usefully compared not on accuracy but on the structure of their cost. A handcrafted feature set requires problem-specific design, the implementation of an extractor for each input format, and empirical validation, often through feature-selection or instance-space analysis. This is a substantial up-front investment in domain expertise and engineering effort, carried out once per problem, after which the per-instance computation is negligible. Latent representations reverse this profile: the same pretrained LLM can be applied to all four problems in our study without modification, and no problem-specific feature engineering is required, but each instance carries a non-negligible inference cost.

The two approaches are therefore best suited to different situations. Handcrafted features amortize their development cost when a problem is studied repeatedly, when its input formats are stable, and when the deployment scale makes per-instance cost the dominant term. Latent representations are more attractive when algorithm selection is required for new or non-standard problems, when instances arrive in heterogeneous formats, or when the infrastructure for handcrafted feature engineering is not yet in place.

Thus, LLM-derived representations should not be viewed as drop-in replacements for exact feature extractors. Their potential value lies in a different regime: when handcrafted extractors are unavailable, expensive to design, difficult to maintain across heterogeneous formats, or when a reusable representation is needed before problem-specific feature engineering has been carried out.

## 6 Conclusions

We investigated how Large Language Models (LLMs) represent Combinatorial Optimization Problems (COPs) and whether these internal representations can support downstream decision-making tasks. Through a combination of *direct querying* and *probing* analyses, we examined both the explicit reasoning capabilities and the implicit knowledge encoded within the models across four benchmark problems, three types of instance representations, and five open-weight models spanning the 3B–120B range.

Two tasks were considered: *feature extraction*, assessing whether models can recover meaningful structural and numerical characteristics from problem instances, and *per-instance algorithm selection*, testing whether internal representations can predict the best solver for each instance.

Our results reveal that:

- The direct querying experiments (Section 4.1) show that LLMs can reliably recover simple, explicitly stated instance features, particularly from natural language and code-like representations. However, performance deteriorates markedly for features requiring even limited computation or aggregation. While increasing model scale improves performance in some settings, these gains are strongly conditioned by feature complexity and representation, and remain limited for more complex features. Explicit Chain-of-Thought (CoT) prompting has a scale-dependent effect: it helps mainly at larger scales and consistently increases computational cost.

- The probing analyses (Section 4.2) show that LLMs encode structural and numerical information in their internal representations, but that this information is not always accessible through direct querying. This supports the distinction between latent decodability and explicit recoverability discussed in Section 5.

- Through an extensive experimental campaign (Section 4.3), we find that LLM-derived representations can serve as effective surrogate instance descriptors for per-instance algorithm selection. Across all evaluated models, their predictive performance is broadly comparable to traditional feature sets, with differences that are small and problem-dependent. In particular, no robust evidence of a systematic advantage emerges for Instance Space Analysis (ISA)-based features over LLM-derived representations.

Taken together, these findings reveal a functional asymmetry between explicit reasoning and internal representation in LLMs. In practical terms, LLM-derived representations are best viewed as complementary descriptors for settings where handcrafted feature engineering is unavailable, costly, or difficult to maintain across heterogeneous formats.

Several directions extend this work. Recent progress in structured reasoning and tool-augmented generation with LLMs offers a complementary route in which exact computation is delegated to deterministic sub-procedures, while the model is responsible for instance interpretation and surrogate representation. A second avenue lies in integrating LLM-derived activations into broader ISA pipelines, using them not only for algorithm selection but also for tasks such as instance clustering, performance landscape modeling, and exploratory analysis of problem spaces. Extending the investigation to other model architectures, larger instance scales, and real-world optimization problems represents a further important direction.

Two limitations of the study deserve to be noted explicitly. The experimental campaign is conducted on a stratified subset of instances drawn from four benchmark COPs, which may affect the stability of some comparative results, particularly in the probing analysis. The model pool spans the 3B–120B range and is restricted to open-weight LLMs for reasons of reproducibility and access to internal activations, leaving closed-weight or substantially larger models outside the present study.

## Acknowledgments

The computational experiments were conducted using the LABIC research infrastructure, supported by funding from the Regione Autonoma Friuli Venezia Giulia, the Italian Ministry of Foreign Affairs, and the Italian Ministry of University and Research.

## Declaration on the Use of Generative Artificial Intelligence

The authors acknowledge the use of ChatGPT and Claude to support language editing, including grammar correction and rephrasing. Following its use, all text was critically reviewed, verified, and revised by the authors, who accept full responsibility for the integrity and accuracy of the manuscript.

## References

[1] W. Wu, M. Ito, Y. Hu, H. Goko, M. Sasaki, M. Yagiura, Heuristic algorithms based on column generation for an online product shipping problem, Comput. Oper. Res. 161 (2024) 106403. doi:10.1016/J.COR.2023.106403.

[2] X. Yu, S. Shen, B. Badri-Koohi, H. Seada, Time window optimization for attended home service delivery under multiple sources of uncertainties, Comput. Oper. Res. 150 (2023) 106045. doi:10.1016/J.COR.2022.106045.

[3] O. X. do Nascimento, T. A. de Queiroz, L. Junqueira, Practical constraints in the container loading problem: Comprehensive formulations and exact algorithm, Comput. Oper. Res. 128 (2021) 105186. doi:10.1016/J.COR.2020.105186.

[4] E. Talbi, Machine learning into metaheuristics: A survey and taxonomy, ACM Comput. Surv. 54 (2022) 129:1–129:32. doi:10.1145/3459664.

[5] A. Matarazzo, R. Torlone, A survey on large language models with some insights on their capabilities and limitations, arXiv, 2025. URL: https://doi.org/10.48550/arXiv.2501.04040.

[6] Z. Fan, B. Ghaddar, X. Wang, L. Xing, Y. Zhang, Z. Zhou, Artificial Intelligence for Operations Research: Revolutionizing the Operations Research Process, arXiv, 2024. URL: https://arxiv.org/abs/2401.03244.

[7] F. Da Ros, L. Di Gaspero, K. Roitero, Probing LLMs on optimization problems: Can they recall and interpret problem features?, in: P. García-Sánchez, E. Hart, S. L. Thomson (Eds.), Applications of Evolutionary Computation - 28th European Conference, EvoApplications 2025, Held as Part of EvoStar 2025, Trieste, Italy, April 23-25, 2025, Proceedings, Part II, volume 15613 of *Lecture Notes in Computer Science*, Springer, 2025, pp. 353–371. doi:10.1007/978-3-031-90065-5_22.

[8] P. Kumar, Large language models (LLMs): survey, technical frameworks, and future challenges, Artif. Intell. Rev. 57 (2024) 260. doi:10.1007/S10462-024-10888-Y.

[9] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang, Y. Wang, W. Ye, Y. Zhang, Y. Chang, P. S. Yu, Q. Yang, X. Xie, A Survey on Evaluation of Large Language Models, ACM Trans. Intell. Syst. Technol. 15 (2024) 39:1–39:45. doi:10.1145/3641289.

[10] E. Meguellati, N. A. Pratama, S. Sadiq, G. Demartini, Are large language models good data preprocessors?, in: G. Long, M. Blumestein, Y. Chang, L. Lewin-Eytan, Z. H. Huang, E. Yom-Tov (Eds.), Companion Proceedings of the ACM on Web Conference 2025, WWW 2025, Sydney, NSW, Australia, 28 April 2025 - 2 May 2025, ACM, 2025, pp. 2129–2132. doi:10.1145/3701716.3717568.

[11] S. Civelli, P. Bernardelle, G. Demartini, The impact of persona-based political perspectives on hateful content detection, in: G. Long, M. Blumestein, Y. Chang, L. Lewin-Eytan, Z. H. Huang, E. Yom-Tov (Eds.), Companion Proceedings of the ACM on Web Conference 2025, WWW 2025, Sydney, NSW, Australia, 28 April 2025 - 2 May 2025, ACM, 2025, pp. 1963–1968. doi:10.1145/3701716.3718383.

[12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, I. Polosukhin, Attention is All you Need, in: I. Guyon, U. von Luxburg, S. Bengio, H. M. Wallach, R. Fergus, S. V. N. Vishwanathan, R. Garnett (Eds.), Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA, 2017, pp. 5998–6008. URL: https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

[13] D. Bahdanau, K. Cho, Y. Bengio, Neural machine translation by jointly learning to align and translate, in: Y. Bengio, Y. LeCun (Eds.), 3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings, 2015. URL: http://arxiv.org/abs/1409.0473.

[14] P. Laban, A. R. Fabbri, C. Xiong, C.-S. Wu, Summary of a haystack: A challenge to long-context LLMs and RAG systems, arXiv, 2024. doi:10.48550/arXiv.2407.01370.

[15] T. Ju, W. Sun, W. Du, X. Yuan, Z. Ren, G. Liu, How large language models encode context knowledge? A layer-wise probing study, in: N. Calzolari, M. Kan, V. Hoste, A. Lenci, S. Sakti, N. Xue (Eds.), Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation, LREC/COLING 2024, 20-25 May, 2024, Torino, Italy, ELRA and ICCL, 2024, pp. 8235–8246. URL: https://aclanthology.org/2024.lrec-main.722.

[16] S. Marks, M. Tegmark, The geometry of truth: Emergent linear structure in large language model representations of true/false datasets, arXiv, 2024. URL: https://arxiv.org/abs/2310.06824.

[17] D. Li, H. Zhao, Q. Zeng, M. Du, Exploring multilingual probing in large language models: A cross-language analysis, arXiv, 2025. URL: https://arxiv.org/abs/2409.14459.

[18] Y. Belinkov, Probing classifiers: Promises, shortcomings, and advances, Comput. Linguistics 48 (2022) 207–219. doi:10.1162/coli_a_00422.

[19] E. Wallace, Y. Wang, S. Li, S. Singh, M. Gardner, Do NLP models know numbers? probing numeracy in embeddings, in: K. Inui, J. Jiang, V. Ng, X. Wan (Eds.), Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, EMNLP-IJCNLP 2019, Hong Kong, China, November 3-7, 2019, Association for Computational Linguistics, 2019, pp. 5306–5314. doi:10.18653/V1/D19-1534.

[20] N. Kim, R. Patel, A. Poliak, A. Wang, P. Xia, R. T. McCoy, I. Tenney, A. Ross, T. Linzen, B. V. Durme, S. R. Bowman, E. Pavlick, Probing what different NLP tasks teach machines about function word comprehension, arXiv, 2019. URL: http://arxiv.org/abs/1904.11544.

[21] R. Huben, H. Cunningham, L. R. Smith, A. Ewart, L. Sharkey, Sparse autoencoders find highly interpretable features in language models, in: The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024, OpenReview.net, 2024. URL: https://openreview.net/forum?id=F76bwRSLeK.

[22] I. Vulic, E. M. Ponti, R. Litschko, G. Glavas, A. Korhonen, Probing pretrained language models for lexical semantics, in: B. Webber, T. Cohn, Y. He, Y. Liu (Eds.), Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, EMNLP 2020, Online, November 16-20, 2020, Association for Computational Linguistics, 2020, pp. 7222–7240. doi:10.18653/V1/2020.EMNLP-MAIN.586.

[23] F. Koto, J. H. Lau, T. Baldwin, Discourse probing of pretrained language models, in: K. Toutanova, A. Rumshisky, L. Zettlemoyer, D. Hakkani-Tür, I. Beltagy, S. Bethard, R. Cotterell, T. Chakraborty, Y. Zhou (Eds.), Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2021, Online, June 6-11, 2021, Association for Computational Linguistics, 2021, pp. 3849–3864. doi:10.18653/V1/2021.NAACL-MAIN.301.

[24] D. Chanin, A. Hunter, O. Camburu, Identifying linear relational concepts in large language models, in: K. Duh, H. Gómez-Adorno, S. Bethard (Eds.), Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), NAACL 2024, Mexico City, Mexico, June 16-21, 2024, Association for Computational Linguistics, 2024, pp. 1524–1535. doi:10.18653/V1/2024.NAACL-LONG.85.

[25] M. Alleman, J. Mamou, M. A. D. Rio, H. Tang, Y. Kim, S. Chung, Syntactic perturbations reveal representational correlates of hierarchical phrase structure in pretrained language models, in: A. Rogers, I. Calixto, I. Vulic, N. Saphra, N. Kassner, O. Camburu, T. Bansal, V. Shwartz (Eds.), Proceedings of the 6th Workshop on Representation Learning for NLP, RepL4NLP@ACL-IJCNLP 2021, Online, August 6, 2021, Association for Computational Linguistics, 2021, pp. 263–276. doi:10.18653/V1/2021.REPL4NLP-1.27.

[26] M. Beloucif, C. Biemann, Probing pre-trained language models for semantic attributes and their values, in: M. Moens, X. Huang, L. Specia, S. W. Yih (Eds.), Findings of the Association for Computational Linguistics: EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 16-20 November, 2021, Association for Computational Linguistics, 2021, pp. 2554–2559. doi:10.18653/V1/2021.FINDINGS-EMNLP.218.

[27] Y. Wang, K. Li, Large Language Models and Operations Research: A Structured Survey, arXiv, 2025. URL: https://arxiv.org/abs/2509.18180.

[28] F. Da Ros, M. Soprano, L. Di Gaspero, K. Roitero, Large Language Models for Combinatorial Optimization: A Systematic Review, arXiv, 2025. URL: https://arxiv.org/abs/2507.03637.

[29] R. Ramamonjison, T. T. L. Yu, R. Li, H. Li, G. Carenini, B. Ghaddar, S. He, M. Mostajabdaveh, A. Banitalebi-Dehkordi, Z. Zhou, Y. Zhang, NL4Opt Competition: Formulating Optimization Problems Based on Their Natural

Language Descriptions, in: M. Ciccone, G. Stolovitzky, J. Albrecht (Eds.), NeurIPS 2022 Competition Track, November 28 - December 9, 2022, Online, volume 220 of *Proceedings of Machine Learning Research*, PMLR, 2021, pp. 189–203. URL: https://proceedings.mlr.press/v220/ramamonjison22a.html.

[30] X. Pan, J. Fang, F. Wu, S. Zhang, Y. Hu, S. Li, X. Li, Guiding Large Language Models in Modeling Optimization Problems via Question Partitioning, in: Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, IJCAI 2025, Montreal, Canada, August 16-22, 2025, ijcai.org, 2025, pp. 2657–2665. doi:10.24963/IJCAI.2025/296.

[31] F. Liu, Y. Yao, P. Guo, Z. Yang, Z. Zhao, X. Lin, X. Tong, M. Yuan, Z. Lu, Z. Wang, Q. Zhang, A Systematic Survey on Large Language Models for Algorithm Design, arXiv, 2024. URL: https://doi.org/10.48550/arXiv.2410.14716.

[32] K. Michailidis, D. C. Tsouros, T. Guns, CP-Bench: Evaluating Large Language Models for Constraint Modelling, arXiv, 2025. URL: https://doi.org/10.48550/arXiv.2506.06052.

[33] D. C. Tsouros, H. Verhaeghe, S. Kadioglu, T. Guns, Holy Grail 2.0: From Natural Language to Constraint Models, arXiv, 2023. URL: https://doi.org/10.48550/arXiv.2308.01589.

[34] A. AhmadiTeshnizi, W. Gao, M. Udell, OptiMUS: Scalable Optimization Modeling with (MI)LP Solvers and Large Language Models, in: Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024, OpenReview.net, 2024, pp. 1–20. URL: https://openreview.net/forum?id=YT1dtdLvSN.

[35] B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli, A. Fawzi, Mathematical discoveries from program search with large language models, Nat. 625 (2024) 468–475. doi:10.1038/S41586-023-06924-6.

[36] K. Sim, Q. Renau, E. Hart, Beyond the hype: Benchmarking LLM-evolved heuristics for bin packing, in: P. García-Sánchez, E. Hart, S. L. Thomson (Eds.), Applications of Evolutionary Computation - 28th European Conference, EvoApplications 2025, Held as Part of EvoStar 2025, Trieste, Italy, April 23-25, 2025, Proceedings, Part II, volume 15613 of *Lecture Notes in Computer Science*, Springer, 2025, pp. 386–402. doi:10.1007/978-3-031-90065-5_24.

[37] N. van Stein, T. Bäck, LLaMEA: A Large Language Model Evolutionary Algorithm for Automatically Generating Metaheuristics, IEEE Trans. Evol. Comput. 29 (2025) 331–345. doi:10.1109/TEVC.2024.3497793.

[38] C. C. Sartori, C. Blum, Combinatorial optimization for all: Using LLMs to aid non-experts in improving optimization algorithms, arXiv, 2025. URL: https://doi.org/10.48550/arXiv.2503.10968.

[39] C. C. Sartori, C. Blum, G. Ochoa, Large Language Models for the Automated Analysis of Optimization Algorithms, in: X. Li, J. Handl (Eds.), Proceedings of the Genetic and Evolutionary Computation Conference, GECCO 2024, Melbourne, VIC, Australia, July 14-18, 2024, ACM, 2024, pp. 160–168. doi:10.1145/3638529.3654086.

[40] C. C. Sartori, C. Blum, G. Ochoa, STNWeb: A new visualization tool for analyzing optimization algorithms, Softw. Impacts 17 (2023) 100558. doi:10.1016/J.SIMPA.2023.100558.

[41] N. van Stein, A. V. Kononova, L. Kotthoff, T. Bäck, Code Evolution Graphs: Understanding Large Language Model Driven Design of Algorithms, in: B. Filipic (Ed.), Proceedings of the Genetic and Evolutionary Computation Conference, GECCO 2025, NH Malaga Hotel, Malaga, Spain, July 14-18, 2025, ACM, 2025, pp. 943–951. doi:10.1145/3712256.3726328.

[42] N. van Stein, H. Yin, A. V. Kononova, T. Bäck, G. Ochoa, Behaviour space analysis of LLM-driven metaheuristic discovery, arXiv, 2025. URL: https://doi.org/10.48550/arXiv.2507.03605.

[43] D. H. Wolpert, W. G. Macready, No free lunch theorems for optimization, IEEE Trans. Evol. Comput. 1 (1997) 67–82. doi:10.1109/4235.585893.

[44] J. R. Rice, The algorithm selection problem, Adv. Comput. 15 (1976) 65–118. doi:10.1016/S0065-2458(08)60520-3.

[45] L. Xu, F. Hutter, H. H. Hoos, K. Leyton-Brown, SATzilla: Portfolio-based Algorithm Selection for SAT, J. Artif. Intell. Res. 32 (2008) 565–606. doi:10.1613/JAIR.2490.

[46] H. Shavit, H. H. Hoos, Revisiting SATZilla Features in 2024, in: S. Chakraborty, J. R. Jiang (Eds.), 27th International Conference on Theory and Applications of Satisfiability Testing, SAT 2024, August 21-24, 2024, Pune, India, volume 305 of *LIPIcs*, Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024, pp. 27:1–27:26. doi:10.4230/LIPICS.SAT.2024.27.

[47] K. Smith-Miles, M. A. Muñoz, Instance Space Analysis for Algorithm Testing: Methodology and Software Tools, ACM Comput. Surv. 55 (2023) 255:1–255:31. doi:10.1145/3572895.

[48] C. Liu, K. Smith-Miles, T. Wauters, A. M. Costa, Instance space analysis for 2d bin packing mathematical models, Eur. J. Oper. Res. 315 (2024) 484–498. doi:10.1016/J.EJOR.2023.12.008.

[49] A. D. Coster, N. Musliu, A. Schaerf, J. Schoisswohl, K. Smith-Miles, Algorithm selection and instance space analysis for curriculum-based course timetabling, J. Sched. 25 (2022) 35–58. doi:10.1007/S10951-021-00701-X.

[50] M. Alissa, K. Sim, E. Hart, Automated Algorithm Selection: from Feature-Based to Feature-Free Approaches, J. Heuristics 29 (2023) 1–38. doi:10.1007/S10732-022-09505-4.

[51] A. Pellegrino, Ö. Akgün, N. Dang, Z. Kiziltan, I. Miguel, Transformer-Based Feature Learning for Algorithm Selection in Combinatorial Optimisation, in: M. G. de la Banda (Ed.), 31st International Conference on Principles and Practice of Constraint Programming, CP 2025, August 10-15, 2025, Glasgow, Scotland, volume 340 of *LIPIcs*, Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025, pp. 31:1–31:22. doi:10.4230/LIPICS.CP.2025.31.

[52] G. Cenikj, G. Petelin, T. Eftimov, TransOptAS: Transformer-Based Algorithm Selection for Single-Objective Optimization, in: X. Li, J. Handl (Eds.), Proceedings of the Genetic and Evolutionary Computation Conference Companion, GECCO 2024, Melbourne, VIC, Australia, July 14-18, 2024, ACM, 2024, pp. 403–406. doi:10.1145/3638530.3654191.

[53] A. Ahmeti, N. Musliu, Hybridizing constraint programming and meta-heuristics for multi-mode resource-constrained multiple projects scheduling problem, J. Heuristics 31 (2025) 1–37. doi:10.1007/S10732-024-09540-3.

[54] L. Kletzander, N. Musliu, Hyper-heuristics for personnel scheduling domains, Artif. Intell. 334 (2024) 104172. doi:10.1016/J.ARTINT.2024.104172.

[55] S. Ceschia, M. Gansterer, S. Mancini, A. Meneghetti, Solving the Online On-Demand Warehousing Problem, Comput. Oper. Res. 170 (2024) 106760. doi:10.1016/J.COR.2024.106760.

[56] K. A. Smith-Miles, D. Baatar, B. J. Wreford, R. Lewis, Towards objective measures of algorithm performance across instance space, Computers & Operations Research 45 (2014) 12–24. doi:10.1016/j.cor.2013.11.015.

[57] K. A. Smith-Miles, R. J. W. James, J. W. Giffin, Y. Tu, A knowledge discovery approach to understanding relationships between scheduling problem structure and heuristic performance, in: Learning and Intelligent Optimization, volume 5851 of *Lecture Notes in Computer Science*, Springer, 2009, pp. 89–103. doi:10.1007/978-3-642-11169-3_7.

[58] K. Smith-Miles, J. Christiansen, M. A. Muñoz, Revisiting *where are the hard knapsack problems?* via Instance Space Analysis, Comput. Oper. Res. 128 (2021) 105184. doi:10.1016/J.COR.2020.105184.

[59] K. Smith-Miles, S. Bowly, Generating new test instances by evolving in instance space, Comput. Oper. Res. 63 (2015) 102–113. doi:10.1016/J.COR.2015.04.022.

[60] K. Liu, Using Instance Space Analysis to Study the Bin Packing Problem, Master Thesis, 2020. URL: https://matilda.unimelb.edu.au/matilda/matildadata/bin_packing/bin_packing.pdf.

[61] DIMACS, The second dimacs international algorithm implementation challenge: General information, 1992. URL: http://archive.dimacs.rutgers.edu/pub/challenge/gen_info.tex, accessed on 26 Oct 2025.

[62] S. Strassl, N. Musliu, Instance space analysis and algorithm selection for the job shop scheduling problem, Comput. Oper. Res. 141 (2022) 105661. doi:10.1016/J.COR.2021.105661.

[63] Minizinc, The minizinc examples archive - v1.0.0, 2017. URL: https://github.com/MiniZinc/minizinc-examples/tree/master, accessed on 26 Oct 2025.

[64] Y. Tang, Y. Yang, Pooling and attention: What are effective designs for LLM-based embedding models?, 2024. URL: https://arxiv.org/abs/2409.02727. arXiv:2409.02727.

[65] W.-Y. Ku, J. C. Beck, Mixed integer programming models for job shop scheduling: A computational analysis, Computers & Operations Research 73 (2016) 165–173. doi:10.1016/j.cor.2016.04.006.

# A Case Studies Specifications

In this appendix, we provide additional details on the case studies used in this paper. Specifically, we report problem formulations, feature definitions, instance formats, and worked examples of the direct-querying prompt for the three problems not already illustrated in the main text. When notation is employed, it is contextual and specific to each problem.

The Graph Coloring Problem (GCP) is illustrated directly in Section 3. For the remaining three problems, we report below an instantiation of the standard direct-querying prompt template introduced in Section 3.2, with one feature chosen per problem to illustrate how the placeholders are populated. The Chain-of-Thought variant differs only in the output instructions, as already shown in the main text. The associated structured-output schema, common to all problems and instantiated below, constrains the model output to a single JSON object whose `value` field carries the predicted feature.

```
{
  "type": "object",
  "additionalProperties": false,
  "required": ["value"],
  "properties": {
    "value": { "anyOf": [ { "type": "[feature-type]" }, { "type": "null" } ] }
  }
}
```

For the Chain-of-Thought variant, the same schema is extended with the additional required field `"reasoning":{"type":"string"}`.

## A.1 Bin Packing Problem

### A.1.1 Problem Formulation

The Bin Packing Problem (BPP) considers a set of items $\mathcal{I} = \{1, \dots n\}$, with $|\mathcal{I}| = n$, each with its own weight ($w_i$), and an infinite number of bins, all with the same capacity ($C$). The objective is to minimize the total number of bins used, subject to the constraints that every item is placed in a bin, and the sum of the weights of the items in each bin does not exceed the capacity.

A possible mathematical formulation for the BPP is as follows:

$$\min \quad \sum_{j \in \mathcal{I}} y_j \tag{2}$$

$$\text{s.t.} \tag{3}$$

$$\sum_{i \in \mathcal{I}} w_i x_{ij} \leq C y_j \qquad \forall j \in \mathcal{I} \tag{4}$$

$$\sum_{j \in \mathcal{I}} x_{ij} = 1 \qquad \forall i \in \mathcal{I} \tag{5}$$

$$y_j \in \{0, 1\} \qquad \forall j \in \mathcal{I} \tag{6}$$

$$x_{ij} \in \{0, 1\} \qquad \forall i \in \mathcal{I} \quad \forall j \in \mathcal{I} \tag{7}$$

$$\tag{8}$$

The model is defined using two decision variables: $y_j$, indicating whether bin $j$ is used, and $x_{ij}$, indicating whether item $i$ is assigned to bin $j$. We assume that at most $n$ bins are available. Objective (2) minimizes the number of used bins. Constraint (4) limits the total weight of the items assigned to each bin by its capacity. Constraint (5) enforces that each item is assigned to exactly one bin. Finally, constraints (6) and (7) define the domains of the decision variables.

### A.1.2 Features

The features considered are as follows:

- **Bin capacity**: The capacity of each bin. This corresponds to $C$ and is directly provided in the instances; therefore, it is an easy feature (○).

- **Items**: The number of items to be assigned to the bins. This corresponds to $n$ and is directly provided in the instances; therefore, it is an easy feature (○).
- **Item weight**: Metrics obtained by aggregating the item weights $w_i$, $i \in \mathcal{I}$. The associated complexity depends on the aggregation: minimum and maximum are mid-complexity features (◓), as they require ordering operations, whereas the average is a hard feature (●), as it requires explicit computation. They are calculated as:

$$w_{\min} = \min_{i \in \mathcal{I}} w_i \tag{9}$$

$$w_{\max} = \max_{i \in \mathcal{I}} w_i, \text{and} \tag{10}$$

$$w_{\text{avg}} = \frac{1}{n} \sum_{i \in \mathcal{I}} w_i \tag{11}$$

### A.1.3 Instance Formats

The standard format for BPP instances is as follows:

> The first line reports the number of items (i.e., the number of weights in the following), the second line reports the capacity of the bins. The subsequent lines report the weights of the items, in descending order.

Thus, an instance in this representation is illustrated below:

```
1000
1000
700
700
700
700
...
```

In this case, the instance considers 1,000 items. The bins have a capacity of 1,000. The first element has a weight of 700, etc. The same instance introduced in natural language for is:

> The instance regards 1000 items. Each bin has a capacity of 1000. The weights of the items are as follows. Note that they are ordered in descending order. Item 1 has a weight of 700. ...

The correspondent code-like format is:

```
int: items = 1000;
array[1..items] of int: weights = [700,700,...];
int: bin_capacity = 1000;
```

### A.1.4 Prompt Example

The example below shows the standard direct-querying prompt instantiated for the BPP, targeting the maximum item weight (a ◓ feature) on a small instance in the standard format. The same template is used for all features defined for this problem; only the `feature_name`, `feature_description`, and `feature_type` placeholders change accordingly.

```
You are given an instance of a combinatorial optimization problem: Bin Packing
Problem.
This is not a coding task. Do not return any code.
Extract the numeric value of the following feature from the instance:
- Feature name: feat_max_weight
- Feature description: The maximum weight among all the items.
- Expected type: integer
The instance is provided here: """
1000
1000
700
700
700
700
...
"""
Instructions:
- Return a JSON object only, with a single field "value", i.e., { "value": ... }.
- The "value" field should contain the numeric value of the required feature
  and should be of the expected type (i.e., integer).
- If the value is unknown/undeterminable, return { "value": null }.
- No explanations, no extra fields.
Answer:
```

## A.2 Jobshop Scheduling Problem

### A.2.1 Problem Formulation

The Jobshop Scheduling Problem (JSP) involves a finite set of jobs (i.e., $\mathcal{J} = \{1, \ldots, n\}$, with $|\mathcal{J}| = n$) and machines (i.e., $\mathcal{M} = \{1, \ldots, m\}$, with $|\mathcal{M}| = m$). Each job consists of a sequence of operations that must be processed on the machines in a given order (indicated as $(\sigma_1^i, \ldots, \sigma_m^i)$ for job $i$), with each job-machine pair associated with a predefined processing time, indicated as $p_{i,j}$. The objective is to construct a feasible schedule that minimizes the makespan. A valid schedule must satisfy three main constraints. First, the prescribed order of operations within each job must be respected. Second, machines cannot process more than one operation at the same time, meaning that no overlaps are allowed. Third, operations are non-preemptive: once an operation has started on a machine, it must run to completion without being interrupted or paused.

A possible mathematical formulation for the JSP is as follows [65]:

$$\min \quad C_{\max} \tag{12}$$

$$\text{s.t.} \tag{13}$$

$$x_{\sigma_h^i i} \geq x_{\sigma_{h-1}^i i} + p_{\sigma_{h-1}^i i} \qquad \forall i \in \mathcal{J} \quad \forall h \in \{1, \ldots, m\} \tag{14}$$

$$x_{ji} \geq x_{jk} + p_{jk} - V \cdot z_{jik} \qquad \forall i \in \mathcal{J} \quad \forall k \in \mathcal{J} \quad \forall j \in \mathcal{M} \quad i < k \tag{15}$$

$$x_{jk} \geq x_{ji} + p_{ji} - V \cdot (1 - z_{jik}) \qquad \forall i \in \mathcal{J} \quad \forall k \in \mathcal{J} \quad \forall j \in \mathcal{M} \quad i < k \tag{16}$$

$$C_{\max} \geq x_{\sigma_m^i i} + p_{\sigma_m^i i} \qquad \forall i \in \mathcal{J} \tag{17}$$

$$x_{ji} \geq 0 \qquad \forall i \in \mathcal{J} \quad \forall j \in \mathcal{M} \tag{18}$$

$$z_{jik} \in \{0, 1\} \qquad \forall i \in \mathcal{J} \quad \forall k \in \mathcal{J} \quad \forall j \in \mathcal{M} \tag{19}$$

The disjunctive model uses a decision variable $x_{ji}$ representing the start time of job $i$ on machine $j$, and a binary variable $z_{jik}$ indicating whether job $i$ is processed before job $k$ on machine $j$. Objective (12) minimizes the makespan. Constraint (14) enforces the technological precedence relations, ensuring that the operations of each job are processed in the correct order. The disjunctive constraints (15) and (16) guarantee that no two jobs are processed on the same machine at the same time; the constant $V$ is chosen to be sufficiently large. Constraint (17) ensures that the makespan is at least as large as the completion time of the last operation of each job. Finally, constraints (18) and (19) define the domains of the decision variables.

### A.2.2 Features

The features considered are as follows:

- **Jobs**: The number of jobs $n$. This information is directly provided in the instance; therefore, it is an easy feature (○).
- **Machines**: The number of machines $m$. This information is directly provided in the instance; therefore, it is an easy feature (○).
- **Operations**: The product of the number of jobs $n$ and the number of machines $m$. This is considered a mid-complexity feature (◓), as it depends only on two elements that are directly provided in the instance. It is calculated as $n \cdot m$.
- **Duration**: Metrics obtained by aggregating the processing times $p_{i,j}$ of job $i \in \mathcal{I}$ on machine $j \in \mathcal{M}$. The associated complexity depends on the aggregation: minimum and maximum are mid-complexity features (◓), as they require ordering operations, whereas the average is a hard feature (●), as it requires explicit computation. They are calculated as follows:

$$p_{\min} = \min_{i \in \mathcal{I},\, j \in \mathcal{M}} p_{i,j}, \tag{20}$$

$$p_{\max} = \max_{i \in \mathcal{I},\, j \in \mathcal{M}} p_{i,j}, \tag{21}$$

$$p_{\text{avg}} = \frac{1}{n \cdot m} \sum_{i \in \mathcal{I}} \sum_{j \in \mathcal{M}} p_{i,j}. \tag{22}$$

### A.2.3 Instance Formats

The standard format for JSP instances is as follows:

> The first line contains two integers: the number of jobs and the number of machines. This defines the overall structure of the problem. Each subsequent line corresponds to one job and lists all the operations that make up that job. Every job consists of a fixed number of operations equal to the number of machines. Each operation is described by a pair of integers: the machine on which the operation must be processed, and the duration (processing time) of the operation on that machine.

Thus, an instance in this representation is illustrated below:

```
10 10
4 88 8 68 6 94 5 99 1 67 2 89 9 77 7 99 0 86 3 92
5 72 3 50 6 69 4 75 2 94 8 66 0 92 1 82 7 94 9 63
9 83 8 61 0 83 1 65 6 64 5 85 7 78 4 85 2 55 3 77
...
```

The same instance introduced in natural language for is:

> The job shop consists of 10 machines. There are 10 jobs to be scheduled. Regarding job 1, it must undergo the following operations: operation 4 with duration 88, ...

The correspondent code-like format is:

```
int: n_machines = 10;;
int: n_jobs = 10;
int: n_tasks_per_job = n_machines;
set of int: jobs = 1..n_jobs;
set of int: tasks = 1..n_tasks_per_job;
array [jobs, tasks] of 0..(n_machines-1): job_task_machine = [4, 8, 6 ... ];
array [jobs, tasks] of int: job_task_duration = [88, 68, 94 ... ];
```

#### A.2.4 Prompt Example

The example below shows the standard direct-querying prompt instantiated for the JSP, targeting the average operation duration (a ● feature) on a small instance in the standard format.

```
You are given an instance of a combinatorial optimization problem: Job Shop
Scheduling Problem.
This is not a coding task. Do not return any code.
Extract the numeric value of the following feature from the instance:
- Feature name: feat_avg_duration
- Feature description: Average duration of all the tasks.
- Expected type: float
The instance is provided here: """
10  10
4  88  8  68  6  94  5  99  1  67  2  89  9  77  7  99  0  86  3  92
5  72  3  50  6  69  4  75  2  94  8  66  0  92  1  82  7  94  9  63
9  83  8  61  0  83  1  65  6  64  5  85  7  78  4  85  2  55  3  77
...
"""
Instructions:
- Return a JSON object only, with a single field "value", i.e., { "value": ... }.
- The "value" field should contain the numeric value of the required feature
  and should be of the expected type (i.e., float).
- If the value is unknown/undeterminable, return { "value": null }.
- No explanations, no extra fields.
Answer:
```

### A.3 Knapsack Problem

#### A.3.1 Problem Formulation

The Knapsack Problem (KP) considers a finite set of items $\mathcal{I} = \{1, \ldots, n\}$, with $|\mathcal{I}| = n$, each with a profit ($p_i$) and a weight ($w_i$), and a knapsack capacity ($C$). The goal is to choose items to be included in the knapsack that maximize total profit subject to the capacity constraint. Variants of KP include the *0–1* KP, where each item can be selected at most once, the *bounded* KP, where each item can be selected at most a given number of times, and the *unbounded* KP, where there is no upper limit on the number of times each item can be selected.

A possible mathematical formulation for the *0–1* KP is as follows:

$$\max \quad \sum_{i \in \mathcal{I}} p_i x_i \tag{23}$$

$$\text{s.t.} \tag{24}$$

$$\sum_{i \in \mathcal{I}} w_i x_i \leq C \tag{25}$$

$$x_i \in \{0, 1\} \qquad \forall i \in \mathcal{I} \tag{26}$$

The model is defined using a binary decision variable $x_i$, which indicates whether item $i$ is selected. Objective (23) maximizes the total profit. Constraint (25) enforces that the total weight of the selected items does not exceed the knapsack capacity $C$, while constraint (26) defines the domain of the decision variables.

### A.3.2 Features

The features considered are as follows:

- **Capacity**: The capacity $C$ of the knapsack. This information is directly provided in the instance; therefore, it is an easy feature (○).
- **Weight**: Metrics obtained by aggregating the item weights $w_i$, $i \in \mathcal{I}$. The associated complexity depends on the aggregation: minimum and maximum are mid-complexity features (◓), as they require ordering operations, whereas the average is a hard feature (●), as it requires explicit computation. They are calculated as follows:

$$w_{\min} = \min_{i \in \mathcal{I}} w_i \tag{27}$$

$$w_{\max} = \max_{i \in \mathcal{I}} w_i, \text{and} \tag{28}$$

$$w_{\text{avg}} = \frac{1}{n} \sum_{i \in \mathcal{I}} w_i \tag{29}$$

- **Profit**: Metrics obtained by aggregating the item weights $p_i$, $i \in \mathcal{I}$. The associated complexity depends on the aggregation: minimum and maximum are mid-complexity features (◓), as they require ordering operations, whereas the average is a hard feature (●), as it requires explicit computation. They are calculated as follows:

$$p_{\min} = \min_{i \in \mathcal{I}} p_i \tag{30}$$

$$p_{\max} = \max_{i \in \mathcal{I}} p_i, \text{and} \tag{31}$$

$$p_{\text{avg}} = \frac{1}{n} \sum_{i \in \mathcal{I}} p_i \tag{32}$$

- **Average efficiency**: Efficiency is calculated as the ratio of an items profit to its weight. It is a hard feature (●), as it requires explicit computation. It is calculated as follows

$$e_{\text{avg}} = \frac{1}{n} \sum_{i \in \mathcal{I}} \frac{p_i}{w_i} \tag{33}$$

- **Weight/profit/efficiency of item 1**: Characteristics of the first item, these are mid-complexity features (◓). They are $w_1$, $p_1$, and $e_1 = p_i/w_i$.

### A.3.3 Instance Formats

The standard format for KP instances is as follows:

> The instance is encoded as follows: the first row reports the name of the instance, and the second row provides the instance class or a placeholder. The third row specifies the maximum capacity of the knapsack. After that, each subsequent row represents an item, where the first value indicates its weight and the second its profit

Thus, an instance in this representation is illustrated below:

```
AlmostStrong001
AlmostStronglyCorrelated
2593
777 928
701 842
552 715
855 1025
109 254
...
```

The same instance introduced in natural language for is:

> The instance is named AlmostStrong001 (AlmostStronglyCorrelated). The knapsack has a capacity of 2593. You have 1000 items to choose from. Item 1 has a profit of 928 and a weight of 777. ...

The correspondent code-like format is:

```
int: n = 1000;
int: weight_max = 2593;
array[1..n] of int: weights = [777, 701, ...];
array[1..n] of int: profits = [928, 842, ...];
```

### A.3.4 Prompt Example

The example below shows the standard direct-querying prompt instantiated for the KP, targeting the average item efficiency (a ● feature, defined as the average ratio of profit to weight) on a small instance in the standard format.

```
You are given an instance of a combinatorial optimization problem: 0-1 Knapsack
Problem.
This is not a coding task. Do not return any code.
Extract the numeric value of the following feature from the instance:
- Feature name: feat_avg_efficiency
- Feature description: The average efficiency of all the items available to
  choose from when building your knapsack. Efficiency is calculated as the
  ratio of an item's profit to its weight.
- Expected type: float
The instance is provided here: """
AlmostStrong001
AlmostStronglyCorrelated
2593
777  928
701  842
552  715
855  1025
109  254
...
"""
Instructions:
- Return a JSON object only, with a single field "value", i.e., { "value": ... }.
- The "value" field should contain the numeric value of the required feature
  and should be of the expected type (i.e., float).
- If the value is unknown/undeterminable, return { "value": null }.
- No explanations, no extra fields.
Answer:
```

## B Direct-Querying: Scaling Regression and Inference Cost

### Scaling regression on accuracy and abstention metrics

Section 4.1 reports direct-querying performance in terms of the WITHIN 1% accuracy for clarity. To verify that the observed scaling patterns are not specific to that metric, we replicate the analysis on three additional indicators: Mean Absolute Error (MAE), exact-match accuracy (EQUALS) and the abstention rate (NULL RATE). For each metric and each (representation, feature-complexity) cell, we fit a linear regression of the metric against $\log_2(b)$, where $b$ is the model size in billions of parameters, using the five baseline configurations (Llama-3.2-3B, Llama-3.1-8B, Llama-3.3-70B, gpt-oss-20b, gpt-oss-120b). MAE slopes are reported in normalized form, dividing the raw slope by the mean MAE within the same cell; this makes them dimensionless and comparable across rows, and a value of $-0.5$ corresponds to the average MAE of the cell halving for each parameter doubling. Slopes for the remaining metrics are reported in their natural scale (accuracy points or abstention probability per parameter doubling), since they already lie in the unit interval and need no further normalization.

Table 5 reports slope, $R^2$ and $p$-value for the three metrics. A negative slope on MAE indicates that prediction error decreases as model size doubles, while a positive slope on WITHIN 1% indicates accuracy improvements at the same rate; the two should therefore move in opposite directions. NULL RATE slopes are interpreted on their own: a positive slope captures an increasing tendency to abstain as scale grows.

The pattern is consistent across the three metrics: WITHIN 1% slopes are non-negative everywhere and reach statistical significance ($p < 0.05$) on the ◓–code-like and ◓–natural language cells, while the corresponding MAE slopes are negative on the same cells with $R^2$ above $0.65$. MAE normalization makes the magnitudes directly comparable: most cells see the average error shrink by between $40\%$ and $60\%$ for each parameter doubling, with the notable exception of standard–○ ($-20\%$), where increased capacity alone appears to compensate less effectively for the rigid input format. Cells with degenerate fits correspond to configurations where direct querying is already saturated across the five model sizes (typical for ○ features under natural language representation). NULL RATE exhibits a different shape: the slope is essentially zero or weakly positive for ○ and ◓ features, and becomes the largest of the three at the ● level (around $+0.07$ per parameter doubling, $R^2 \approx 0.25$ across the three representations). This is the regression-level counterpart of the abstention pattern discussed in Section 4.1: larger reasoning-oriented models do not necessarily improve their ● accuracy, but they do increase their tendency to abstain.

### Inference cost reference

Table 6 reports the mean generation time per feature–instance, broken down by model, prompting mode, and feature complexity. The values are aggregated over problems, representations, and random seeds. They constitute the numeric reference for the cost discussion in Section 4.1 and complement Figures 8 and 9, which provide the corresponding graphical view for individual problems and representations.

The aggregate Chain-of-Thought-to-standard time ratio across feature complexities is $1.87\times$ on Llama-3.2-3B, $1.27\times$ on Llama-3.1-8B, and $1.53\times$ on Llama-3.3-70B, indicating that the smallest model pays the largest relative cost for

Table 5: Scaling regression of three direct-querying metrics on $\log_2$(model size), computed on the baseline (*none* or *native*) configurations across the five evaluated models. Each cell reports slope, $R^2$ and $p$-value of the linear fit. The MAE slope is normalized by the cell mean and is therefore dimensionless: a value of $-0.5$ corresponds to the average error in the cell halving for each parameter doubling. *Cells marked with '–' correspond to configurations where the metric is degenerate (e.g., already saturated at the maximum value across all model sizes).*

| | | WITHIN 1% | | | MAE (normalized) | | | NULL RATE | | |
|---|---|---|---|---|---|---|---|---|---|---|
| **Repr.** | **Compl.** | Slope | $R^2$ | $p$ | Slope | $R^2$ | $p$ | Slope | $R^2$ | $p$ |
| standard | ○ | 0.057 | 0.36 | 0.29 | −0.20 | 0.25 | 0.39 | 0.001 | 0.00 | 0.94 |
| | ◓ | 0.074 | 0.79 | 0.04 | −0.47 | 0.67 | 0.09 | 0.014 | 0.07 | 0.66 |
| | ● | 0.059 | 0.58 | 0.14 | −0.40 | 0.76 | 0.06 | 0.079 | 0.27 | 0.38 |
| code-like | ○ | 0.000 | 0.51 | 0.17 | −0.73 | 0.51 | 0.17 | – | – | – |
| | ◓ | 0.083 | 0.90 | 0.01 | −0.59 | 0.81 | 0.04 | 0.010 | 0.05 | 0.71 |
| | ● | 0.033 | 0.80 | 0.04 | −0.46 | 0.76 | 0.05 | 0.073 | 0.26 | 0.38 |
| natural language | ○ | – | – | – | – | – | – | – | – | – |
| | ◓ | 0.046 | 0.90 | 0.01 | −0.46 | 0.40 | 0.25 | 0.005 | 0.02 | 0.84 |
| | ● | 0.041 | 0.61 | 0.12 | −0.50 | 0.77 | 0.05 | 0.072 | 0.25 | 0.39 |

Table 6: Mean generation time per feature–instance (seconds) by model, prompting mode, and feature complexity, averaged over problems and representations. *Llama* models are evaluated under the standard (*none*) and Chain-of-Thought (*prompted*) prompt variants. *gpt-oss* models are evaluated under the native reasoning interface (*native*). The reduced inference time of gpt-oss-120b compared to Llama-3.3-70B reflects the mixture-of-experts architecture, which activates only a fraction of the parameters per token.

| **Model** | **Mode** | ○ | ◓ | ● |
|---|---|---|---|---|
| Llama-3.2-3B | *none* | 0.46 | 0.22 | 0.43 |
| | *prompted* | 0.78 | 0.51 | 0.82 |
| Llama-3.1-8B | *none* | 0.89 | 0.45 | 0.86 |
| | *prompted* | 1.05 | 0.60 | 1.15 |
| Llama-3.3-70B | *none* | 2.28 | 1.22 | 2.16 |
| | *prompted* | 3.15 | 1.99 | 3.52 |
| gpt-oss-20b | *native* | 0.96 | 0.66 | 1.29 |
| gpt-oss-120b | *native* | 0.65 | 0.38 | 0.71 |

explicit reasoning. As discussed in Section 4.1, the same model is also the only one for which Chain-of-Thought consistently degrades accuracy, so the smallest model is doubly penalized: it incurs the highest relative time overhead without any compensating accuracy gain.

# C Feature Probing Results

Tables 7–10 report the MAE obtained for each combination of problem, instance representation, feature complexity, and model. Each table contrasts direct querying (in the standard *none*/*native* mode used as baseline) with probing under the three regressors considered in the study (Linear, LightGBM, MLP). Probing values use the mean pooling strategy as default; the same qualitative patterns hold under max and last pooling, with the per-cell numerical detail available in the companion repository.

Table 7: Direct querying (*none*/*native* mode) and probing (mean pooling) MAE for BPP, broken down by representation, feature complexity, and model. Each cell reports the MAE on the probing validation split. *Cells marked with '–' denote configurations not defined for the problem.*

| | | | standard | | | code-like | | | natural language | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | ○ | ◓ | ● | ○ | ◓ | ● | ○ | ◓ | ● |
| Direct Querying | | Llama 3.2 (3B) | 217.47 | 507.58 | 159.51 | 0.000 | 113.27 | 154.63 | 0.000 | 29.94 | 115.00 |
| | | Llama 3.1 (8B) | 155.68 | 406.20 | 152.56 | 0.000 | 107.02 | 114.90 | 0.000 | 14.38 | 36.54 |
| | | Llama 3.3 (70B) | 307.01 | 87.70 | 22.88 | 0.000 | 0.000 | 41.87 | 0.000 | 0.000 | 23.10 |
| | | GPT-OSS (20B) | 152.12 | 34.37 | – | 0.000 | 0.000 | – | 0.000 | 0.000 | – |
| | | GPT-OSS (120B) | 314.21 | 58.21 | 21.12 | 0.000 | 0.000 | 5.577 | 0.000 | 0.000 | 4.390 |
| Probing | Linear | Llama 3.2 (3B) | 2.289 | 2.960 | 1.265 | 2.963 | 2.901 | 1.376 | 3.947 | 3.235 | 1.549 |
| | | Llama 3.1 (8B) | 3.202 | 2.993 | 1.905 | 3.952 | 3.447 | 1.565 | 4.342 | 3.732 | 2.128 |
| | | Llama 3.3 (70B) | 4.029 | 4.863 | 3.261 | 3.996 | 4.895 | 3.530 | 4.647 | 5.062 | 3.369 |
| | | GPT-OSS (20B) | 4.049 | 4.096 | 2.109 | 4.236 | 4.043 | 2.149 | 4.240 | 4.993 | 4.444 |
| | | GPT-OSS (120B) | 4.165 | 3.405 | 1.866 | 4.032 | 3.890 | 2.207 | 5.455 | 5.743 | 4.023 |
| | LightGBM | Llama 3.2 (3B) | 3.007 | 4.911 | 4.562 | 4.346 | 5.357 | 5.045 | 6.424 | 7.738 | 6.388 |
| | | Llama 3.1 (8B) | 3.399 | 5.531 | 6.013 | 3.580 | 6.887 | 6.995 | 5.647 | 7.850 | 6.942 |
| | | Llama 3.3 (70B) | 5.116 | 6.809 | 7.499 | 4.303 | 9.217 | 9.041 | 4.070 | 8.023 | 6.905 |
| | | GPT-OSS (20B) | 13.69 | 11.57 | 10.48 | 8.290 | 8.924 | 8.675 | 5.704 | 7.929 | 7.825 |
| | | GPT-OSS (120B) | 3.811 | 5.654 | 5.441 | 4.639 | 7.204 | 6.286 | 9.012 | 11.82 | 10.54 |
| | MLP | Llama 3.2 (3B) | 43.93 | 25.36 | 15.23 | 46.53 | 28.60 | 16.92 | 104.79 | 69.31 | 35.75 |
| | | Llama 3.1 (8B) | 43.97 | 28.26 | 18.22 | 61.10 | 34.32 | 21.04 | 117.37 | 80.14 | 46.75 |
| | | Llama 3.3 (70B) | 41.26 | 25.90 | 19.18 | 44.46 | 35.72 | 24.49 | 71.56 | 44.33 | 30.10 |
| | | GPT-OSS (20B) | 60.60 | 36.42 | 24.44 | 62.33 | 37.53 | 27.67 | 120.61 | 89.09 | 69.52 |
| | | GPT-OSS (120B) | 54.96 | 33.02 | 25.67 | 54.32 | 36.87 | 26.84 | 105.58 | 72.90 | 44.51 |

Table 8: Direct querying (*none*/*native* mode) and probing (mean pooling) MAE for GCP, broken down by representation, feature complexity, and model. Each cell reports the MAE on the probing validation split. *Cells marked with '–' denote configurations not defined for the problem.*

| | | | standard | | | code-like | | | natural language | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | ○ | ◓ | ● | ○ | ◓ | ● | ○ | ◓ | ● |
| Direct Querying | | Llama 3.2 (3B) | 941.34 | – | 29.66 | 0.000 | – | 59.14 | 0.000 | – | 220.02 |
| | | Llama 3.1 (8B) | 33.37 | – | 66.89 | 0.000 | – | 38.42 | 0.000 | – | 43.10 |
| | | Llama 3.3 (70B) | 0.000 | – | 10.00 | 0.000 | – | 11.90 | 0.000 | – | 13.06 |
| | | GPT-OSS (20B) | 0.000 | – | 0.004 | 0.000 | – | 0.000 | 0.000 | – | 0.005 |
| | | GPT-OSS (120B) | 0.000 | – | 0.007 | 0.000 | – | 0.064 | 0.000 | – | 0.094 |
| Probing | Linear | Llama 3.2 (3B) | 34.10 | – | 1.974 | 31.06 | – | 1.956 | 20.06 | – | 1.789 |
| | | Llama 3.1 (8B) | 34.29 | – | 1.942 | 31.59 | – | 2.017 | 25.99 | – | 1.857 |
| | | Llama 3.3 (70B) | 29.93 | – | 1.867 | 34.60 | – | 1.949 | 23.86 | – | 1.830 |
| | | GPT-OSS (20B) | 27.14 | – | 1.774 | 28.01 | – | 1.891 | 17.00 | – | 1.674 |
| | | GPT-OSS (120B) | 30.96 | – | 1.787 | 29.54 | – | 1.941 | 24.48 | – | 1.779 |
| | LightGBM | Llama 3.2 (3B) | 68.06 | – | 2.553 | 50.39 | – | 2.429 | 54.94 | – | 2.390 |
| | | Llama 3.1 (8B) | 64.10 | – | 2.581 | 56.60 | – | 2.595 | 54.13 | – | 2.423 |
| | | Llama 3.3 (70B) | 64.34 | – | 2.613 | 63.18 | – | 2.566 | 49.51 | – | 2.418 |
| | | GPT-OSS (20B) | 51.23 | – | 2.301 | 65.45 | – | 2.626 | 47.02 | – | 2.360 |
| | | GPT-OSS (120B) | 57.42 | – | 2.381 | 53.49 | – | 2.592 | 68.97 | – | 2.706 |
| | MLP | Llama 3.2 (3B) | 352.99 | – | 3.668 | 349.82 | – | 3.422 | 356.41 | – | 4.160 |
| | | Llama 3.1 (8B) | 355.10 | – | 3.634 | 352.64 | – | 3.680 | 359.66 | – | 4.060 |
| | | Llama 3.3 (70B) | 346.97 | – | 3.526 | 338.29 | – | 3.410 | 348.24 | – | 3.292 |
| | | GPT-OSS (20B) | 315.46 | – | 3.422 | 349.20 | – | 3.381 | 352.06 | – | 4.356 |
| | | GPT-OSS (120B) | 349.78 | – | 3.525 | 344.19 | – | 3.496 | 349.08 | – | 3.223 |

Table 9: Direct querying (*none*/*native* mode) and probing (mean pooling) MAE for JSP, broken down by representation, feature complexity, and model. Each cell reports the MAE on the probing validation split. *Cells marked with '–' denote configurations not defined for the problem.*

| | | | standard | | | code-like | | | natural language | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | ○ | ◓ | ● | ○ | ◓ | ● | ○ | ◓ | ● |
| Direct Querying | | Llama 3.2 (3B) | 20.20 | 129.03 | 18.26 | 0.000 | 21.97 | 1.791 | 0.000 | 15.98 | 1.452 |
| | | Llama 3.1 (8B) | 17.87 | 89.86 | 12.45 | 0.000 | 11.87 | 0.000 | 0.000 | 71.25 | 0.235 |
| | | Llama 3.3 (70B) | 21.87 | 58.37 | 13.70 | 0.000 | 0.000 | 0.000 | 0.000 | 2.985 | 0.000 |
| | | GPT-OSS (20B) | 0.000 | 0.084 | 0.777 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 |
| | | GPT-OSS (120B) | 0.602 | 4.029 | 2.656 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 |
| Probing | Linear | Llama 3.2 (3B) | 0.597 | 6.392 | 0.000 | 0.535 | 4.844 | 0.000 | 0.522 | 5.734 | 0.000 |
| | | Llama 3.1 (8B) | 0.656 | 9.048 | 0.000 | 0.520 | 4.657 | 0.000 | 0.482 | 6.278 | 0.000 |
| | | Llama 3.3 (70B) | 0.661 | 9.041 | 0.000 | 0.678 | 4.643 | 0.000 | 0.511 | 8.036 | 0.000 |
| | | GPT-OSS (20B) | 0.642 | 7.234 | 0.000 | 0.556 | 2.801 | 0.000 | 0.577 | 3.859 | 0.000 |
| | | GPT-OSS (120B) | 0.565 | 6.215 | 0.000 | 0.598 | 4.122 | 0.000 | 0.689 | 7.645 | 0.000 |
| | LightGBM | Llama 3.2 (3B) | 1.460 | 12.68 | 0.000 | 1.452 | 7.567 | 0.000 | 1.460 | 9.377 | 0.000 |
| | | Llama 3.1 (8B) | 1.203 | 9.401 | 0.000 | 1.359 | 8.357 | 0.000 | 1.034 | 9.382 | 0.000 |
| | | Llama 3.3 (70B) | 1.414 | 6.883 | 0.000 | 1.782 | 9.026 | 0.000 | 1.284 | 10.58 | 0.000 |
| | | GPT-OSS (20B) | 1.871 | 10.83 | 0.000 | 1.737 | 7.027 | 0.000 | 1.274 | 6.880 | 0.000 |
| | | GPT-OSS (120B) | 2.298 | 8.978 | 0.000 | 1.640 | 10.43 | 0.000 | 1.631 | 9.928 | 0.000 |
| | MLP | Llama 3.2 (3B) | 3.360 | 35.36 | 0.016 | 4.050 | 39.08 | 0.011 | 4.133 | 39.13 | 0.007 |
| | | Llama 3.1 (8B) | 3.079 | 30.79 | 0.012 | 3.834 | 56.03 | 0.014 | 3.155 | 33.16 | 0.012 |
| | | Llama 3.3 (70B) | 3.377 | 24.07 | 0.014 | 4.220 | 34.74 | 0.012 | 3.423 | 32.17 | 0.011 |
| | | GPT-OSS (20B) | 4.893 | 32.69 | 0.014 | 4.138 | 37.36 | 0.012 | 6.750 | 38.79 | 0.010 |
| | | GPT-OSS (120B) | 4.397 | 27.91 | 0.016 | 4.256 | 37.59 | 0.018 | 5.016 | 33.94 | 0.010 |

Table 10: Direct querying (*none*/*native* mode) and probing (mean pooling) MAE for KP, broken down by representation, feature complexity, and model. Each cell reports the MAE on the probing validation split. *Cells marked with '–' denote configurations not defined for the problem.*

| | | | standard | | | code-like | | | natural language | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | ○ | ◓ | ● | ○ | ◓ | ● | ○ | ◓ | ● |
| Direct Querying | | Llama 3.2 (3B) | 320.1K | 1.2K | 386.91 | 69.66 | 77.7K | 284.25 | 0.000 | 16.2K | 240.48 |
| | | Llama 3.1 (8B) | 289.8K | 151.43 | 357.94 | 0.000 | 45.9K | 268.54 | 0.000 | 39.9K | 251.33 |
| | | Llama 3.3 (70B) | 317.7K | 139.27 | 76.17 | 0.000 | 12.15 | 55.51 | 0.000 | 1.007 | 43.24 |
| | | GPT-OSS (20B) | 10.6K | 25.76 | 40.53 | 0.000 | 0.000 | 1.985 | 0.000 | 0.000 | 0.860 |
| | | GPT-OSS (120B) | 5.1K | 61.15 | – | 0.000 | 0.005 | 4.100 | 0.000 | 0.000 | 0.000 |
| Probing | Linear | Llama 3.2 (3B) | 40.3K | 55.70 | 6.104 | 33.0K | 70.10 | 7.665 | 28.5K | 70.75 | 5.873 |
| | | Llama 3.1 (8B) | 42.5K | 53.49 | 6.136 | 28.3K | 70.95 | 6.639 | 24.5K | 66.38 | 5.913 |
| | | Llama 3.3 (70B) | 30.7K | 57.81 | 6.354 | 25.1K | 64.04 | 6.812 | 21.9K | 57.19 | 5.971 |
| | | GPT-OSS (20B) | 44.8K | 68.33 | 6.811 | 30.7K | 73.73 | 6.919 | 46.1K | 70.27 | 6.071 |
| | | GPT-OSS (120B) | 34.3K | 63.77 | 4.934 | 33.2K | 65.65 | 5.779 | 34.8K | 63.66 | 5.904 |
| | LightGBM | Llama 3.2 (3B) | 45.8K | 68.99 | 12.10 | 31.2K | 77.88 | 9.217 | 44.3K | 80.37 | 12.49 |
| | | Llama 3.1 (8B) | 51.4K | 68.87 | 11.41 | 33.7K | 77.10 | 9.727 | 30.2K | 77.98 | 10.02 |
| | | Llama 3.3 (70B) | 62.9K | 72.00 | 14.14 | 39.1K | 72.24 | 10.53 | 32.1K | 70.19 | 10.13 |
| | | GPT-OSS (20B) | 69.0K | 78.23 | 14.32 | 30.7K | 77.71 | 8.267 | 49.2K | 77.12 | 10.35 |
| | | GPT-OSS (120B) | 49.0K | 79.12 | 11.48 | 40.0K | 77.96 | 9.788 | 52.8K | 77.32 | 14.79 |
| | MLP | Llama 3.2 (3B) | 317.4K | 111.26 | 52.38 | 316.8K | 106.64 | 53.24 | 316.5K | 109.69 | 55.96 |
| | | Llama 3.1 (8B) | 317.2K | 111.65 | 53.28 | 316.0K | 107.33 | 55.47 | 316.5K | 109.04 | 55.03 |
| | | Llama 3.3 (70B) | 316.4K | 109.24 | 50.84 | 315.6K | 106.81 | 53.52 | 316.4K | 107.80 | 53.91 |
| | | GPT-OSS (20B) | 317.0K | 126.63 | 55.27 | 317.5K | 107.59 | 53.48 | 316.7K | 108.84 | 56.01 |
| | | GPT-OSS (120B) | 316.9K | 112.59 | 53.38 | 316.7K | 107.76 | 53.18 | 317.1K | 111.92 | 58.66 |

# D Complete Set of Statistical Analysis Tables

This appendix reports the full results of the mixed linear models used in Section 4.3 to assess the effects of classifier, feature representation, pooling strategy, and LLM model on algorithm-selection accuracy. All models include random intercepts at the block level, defined as the (*problem*, *replicate*) pair, with fixed-effect estimates reported alongside standard errors, $z$-statistics, $p$-values, and 95% confidence intervals.

- Table 11 reports classifier and model effects on the full pool of observations (Section 4.3, classifier comparison).
- Table 12 compares ISA-based and handcrafted features on the baseline subset (Section 4.3, baseline comparison).
- Table 13 presents the full-factorial mixed model including representation, pooling, classifier, and LLM model as fixed factors (Section 4.3, full-factorial analysis).
- Table 14 reports the per-problem comparison between ISA and the LLM-derived standard representation with LightGBM.

Table 11: Mixed linear model for classifier and model effects, specified as `accuracy ~ C(classifier, ref='MostFrequentClassifier') + C(model, ref='baseline')`, with random intercept at the (*problem*, *replicate*) block level. The *baseline* level of the *model* factor groups handcrafted and ISA feature sets, which are model-independent. The analysis is based on $n = 3760$ observations and $g = 20$ groups.

| **Parameter** | **Coef.** | **Std.Err.** | **z** | **p-value** | **[0.025** | **0.975]** |
|---|---|---|---|---|---|---|
| Intercept (MostFrequentClassifier, baseline) | 0.793 | 0.040 | 19.831 | $< 0.001$ | 0.714 | 0.871 |
| Classifier: LightGBM (vs MostFrequentClassifier) | 0.041 | 0.002 | 20.822 | $< 0.001$ | 0.037 | 0.044 |
| Classifier: LogisticRegression (vs MostFrequentClassifier) | 0.014 | 0.002 | 7.278 | $< 0.001$ | 0.010 | 0.018 |
| Classifier: MLPClassifier (vs MostFrequentClassifier) | $-0.013$ | 0.002 | $-6.435$ | $< 0.001$ | $-0.016$ | $-0.009$ |
| Model: Llama-3.1-8B (vs baseline) | 0.018 | 0.004 | 4.796 | $< 0.001$ | 0.010 | 0.025 |
| Model: Llama-3.2-3B (vs baseline) | 0.019 | 0.004 | 5.114 | $< 0.001$ | 0.012 | 0.026 |
| Model: Llama-3.3-70B (vs baseline) | 0.016 | 0.004 | 4.455 | $< 0.001$ | 0.009 | 0.024 |
| Model: gpt-oss-120b (vs baseline) | 0.019 | 0.004 | 5.155 | $< 0.001$ | 0.012 | 0.026 |
| Model: gpt-oss-20b (vs baseline) | 0.016 | 0.004 | 4.212 | $< 0.001$ | 0.008 | 0.023 |
| *Group variance* ($\sigma^2$) | 0.032 | (0.238) | | | | |

| **Effect** | **Test** | **Statistic (df)** | **p-value** |
|---|---|---|---|
| Classifier (controlling for model) | LRT | $\chi^2(3) = 742.403$ | $1.34 \times 10^{-160}$ |

Table 12: Mixed linear model and post-hoc tests comparing ISA-based and handcrafted features on the baseline subset, specified as `accuracy ~ C(classifier, ref='LogisticRegression') + C(representation, ref='hc')`, with random intercept at the block level. The analysis is based on $n = 80$ observations and $g = 20$ groups.

| **Parameter** | **Coef.** | **Std.Err.** | **z** | **p-value** | **[0.025** | **0.975]** |
|---|---|---|---|---|---|---|
| Intercept (LogisticRegression, Handcrafted) | 0.815 | 0.038 | 21.679 | $< 0.001$ | 0.741 | 0.888 |
| Classifier: LightGBM (vs LogisticRegression) | 0.024 | 0.008 | 2.893 | 0.004 | 0.008 | 0.040 |
| Representation: ISA (vs Handcrafted) | 0.007 | 0.008 | 0.908 | 0.364 | $-0.009$ | 0.023 |
| *Group variance* ($\sigma^2$) | 0.027 | (0.276) | | | | |

| **Comparison** | **Test** | **Statistic (df)** | **p-value / CI** | **Interpretation** |
|---|---|---|---|---|
| ISA vs Handcrafted | LRT | $\chi^2(1) = 0.819$ | $p = 0.365$ | Not significant |
| ISA vs Handcrafted | Tukey HSD | $M_{\text{diff}} = 0.0074$ | $p_{\text{adj}} = 0.848$, $\text{CI}_{95\%} = [-0.069, 0.084]$ | Not significant |
| ISA vs Handcrafted ($\varepsilon = 0.01$) | Wilcoxon TOST | $M_{\text{diff}} = 0.0000$ | $p_{\text{low}} = 0.0063$, $p_{\text{high}} = 0.0500$, $\text{CI}_{95\%} = [-0.0471, 0.0755]$ | Not equivalent |

Table 13: Full-factorial mixed linear model and corresponding likelihood-ratio (LRT) comparisons, specified as `accuracy ~ C(classifier, ref='LogisticRegression') + C(model, ref='Llama-3.2-3B') + C(representation, ref='standard') × C(pooling, ref='mean')`, with random intercept at the (*problem*, *replicate*) block level. Analysis restricted to the two best-performing classifiers (LightGBM and LogisticRegression). The analysis is based on $n = 1800$ observations and $g = 20$ groups.

| **Parameter** | **Coef.** | **Std.Err.** | **z** | **p-value** | **[0.025** | **0.975]** |
|---|---|---|---|---|---|---|
| Intercept (LogisticRegression, Llama-3.2-3B, standard, Mean) | 0.830 | 0.037 | 22.659 | $< 0.001$ | 0.758 | 0.902 |
| Classifier: LightGBM (vs LogisticRegression) | 0.027 | 0.002 | 16.761 | $< 0.001$ | 0.023 | 0.030 |
| Model: Llama-3.1-8B (vs Llama-3.2-3B) | −0.001 | 0.003 | −0.483 | 0.629 | −0.006 | 0.004 |
| Model: Llama-3.3-70B (vs Llama-3.2-3B) | −0.001 | 0.003 | −0.516 | 0.606 | −0.006 | 0.004 |
| Model: gpt-oss-120b (vs Llama-3.2-3B) | −0.002 | 0.003 | −0.604 | 0.546 | −0.006 | 0.003 |
| Model: gpt-oss-20b (vs Llama-3.2-3B) | −0.006 | 0.003 | −2.329 | 0.020 | −0.011 | −0.001 |
| Representation: code-like (vs standard) | −0.005 | 0.003 | −1.605 | 0.108 | −0.012 | 0.001 |
| Representation: natural language (vs standard) | −0.005 | 0.003 | −1.357 | 0.175 | −0.011 | 0.002 |
| Pooling: Last (vs Mean) | −0.010 | 0.003 | −3.091 | 0.002 | −0.017 | −0.004 |
| Pooling: Max (vs Mean) | 0.002 | 0.003 | 0.472 | 0.637 | −0.005 | 0.008 |
| Repr. × Pool: code-like × Last | 0.004 | 0.005 | 0.794 | 0.427 | −0.006 | 0.013 |
| Repr. × Pool: natural language × Last | 0.006 | 0.005 | 1.363 | 0.173 | −0.003 | 0.016 |
| Repr. × Pool: code-like × Max | −0.000 | 0.005 | −0.072 | 0.942 | −0.010 | 0.009 |
| Repr. × Pool: natural language × Max | 0.008 | 0.005 | 1.637 | 0.102 | −0.002 | 0.017 |
| *Group variance* ($\sigma^2$) | 0.027 | (0.253) | | | | |

| **Effect / Comparison** | **Test** | **Statistic (df)** | **p-value** | **Significance** |
|---|---|---|---|---|
| Representation (conditional) | LRT | $\chi^2(6) = 11.353$ | 0.0781 | Not significant |
| Pooling (conditional) | LRT | $\chi^2(6) = 37.389$ | $1.48 \times 10^{-6}$ | **Significant** |
| Representation × Pooling | LRT | $\chi^2(4) = 4.682$ | 0.3215 | Not significant |
| LLM model (conditional) | LRT | $\chi^2(4) = 6.381$ | 0.1724 | Not significant |

Table 14: Per-problem comparison between ISA baseline and the LLM-derived standard/Mean configuration with LightGBM as downstream classifier. The OLS-based test contrasts the baseline mean against the full-factorial mean per problem; the rightmost column reports the corresponding $p$-value of the F-test. Only BPP reaches statistical significance ($p < 0.05$).

| **Problem** | **Mean accuracy (ISA)** | **Mean accuracy (LLM)** | **Δ (LLM−ISA)** | **p-value** |
|---|---|---|---|---|
| BPP | 0.866 | 0.907 | +0.041 | 0.001 |
| GCP | 0.903 | 0.918 | +0.014 | 0.153 |
| JSP | 0.982 | 0.991 | +0.009 | 0.246 |
| KP | 0.611 | 0.586 | −0.025 | 0.358 |

For BPP, the only problem with a significant global difference, the post-hoc Tukey HSD on representation–pooling combinations within the factorial subset does not detect any significant pairwise contrast among the LLM-based configurations, indicating that the global effect is driven by the gap between ISA and the LLM-based pool as a whole rather than by a specific representation–pooling choice.